Social Network Analysis and Mining (2026) 16:25
https://doi.org/10.1007/s13278-025-01552-6

ORIGINAL ARTICLE

A sophisticated feature vectorization-based stacked machine learning approach for fake news detection in Bangla and English

Md. Sabbir Hossen[1] · Fahim Al Farid[2] · Pabon Shaha[1,3] · Md. Mowahibur Rahman Twake[1] · Fahjimatus Sabah[1] · K. M. Mursalin Billah Rezwan[1] · Anichur Rahman[3,4] · Hezerul Abdul Karim[5] · Abu Saleh Musa Miah[6]

Received: 23 May 2025 / Revised: 21 October 2025 / Accepted: 23 October 2025

Abstract

The rapid spread of fake news through social media and the internet poses a major challenge, especially in developing countries like Bangladesh. Fake news, consisting of misleading or fabricated content, can severely impact individuals, organizations, and society. Many existing detection methods are complex and resource-intensive, making them unsuitable for real-time applications or low-resource languages. To address this issue, we proposed an efficient fake news detection system using a stacked machine learning model with TF-IDF for feature extraction. TF-IDF converts text into a numerical representation by emphasizing key terms, while the stacked ensemble model improves classification accuracy by integrating multiple machine learning algorithms. We also explored various machine learning classifiers, as well as the mBERT and Word2Vec feature vectorization techniques. We evaluated our system using three datasets: one English and two Bangla fake news detection datasets. The TF-IDF + Stacking model achieved 99.6% accuracy and a 99.8% F1-score on the English dataset. For the first Bangla dataset, accuracy improved from 74.8 to 85.2% after applying SMOTE. On the second Bangla dataset, BanFakeNews, the model achieved 98.4% accuracy, demonstrating strong performance across languages. This research introduces a lightweight yet highly effective machine learning-based fake news detection system, making it suitable for real-world applications, especially in multilingual and resource-limited settings.

Keywords Machine learning · Fake news · Social media · Natural language processing · Feature extraction · Ensemble method · Deep learning

✉ Anichur Rahman
ar36248@georgiasouthern.edu

✉ Hezerul Abdul Karim
hezerul@mmu.edu.my

✉ Abu Saleh Musa Miah
musa.cse@ru.ac.bd

Md. Sabbir Hossen
sabbir.hossen@bu.edu.bd

Fahim Al Farid
fahmid.farid@gmail.com

Pabon Shaha
pabonshahacse15@gmail.com

Md. Mowahibur Rahman Twake
twake.edu@gmail.com

Fahjimatus Sabah
fahjimatus.sabah@gmail.com

K. M. Mursalin Billah Rezwan
mursalin.cse56@gmail.com

[1] Department of Computer Science and Engineering, Bangladesh University, Mohammadpur, Dhaka 1207, Bangladesh

[2] Faculty of Computer Science and Informatics, Berlin School of Business and Innovation, Karl-Marx-Straße 97-99, Berlin 12043, Germany

[3] School of Computing, Georgia Southern University, Statesboro, GA 30458, USA

[4] Department of Computer Science and Engineering, National Institute of Textile Engineering and Research (NITER), Constituent Institute of the University of Dhaka, Kohinoor Gate, Dhaka-Aricha Hwy, Nayarhat, Savar, Dhaka, Bangladesh

[5] Centre for Image and Vision Computing (CIVC), COE for Artificial Intelligence, Faculty of Artificial Intelligence and Engineering (FAIE), Multimedia University, Cyberjaya 63100, Malaysia

[6] Department of Computer Science and Engineering, The University of Rajshahi, Rajshahi, Bangladesh

Published online: 31 December 2025

1 Introduction

The modern generation has grown up heavily dependent on social media, with platforms like Facebook, Instagram, TikTok, and X (Twitter) having permeated every aspect of daily life. This evolution has been driven by the tremendous development in recent years, and the widespread use of wireless technologies like Wi-Fi, 3G, 4G, and 5G networks (Alghamdi et al. 2024; Rahman et al. 2024). These advancements have revolutionized how people connect with the Internet, making social network platforms among the most widely utilized services globally, accessible to people whenever, wherever (Smaïli et al. 2024). People create profiles on social networking platforms to connect with friends, communicate with them, and publish or tweet content, more people are exposed to false news. Although these platforms provide a wealth of opportunities for communication, amusement, and information exchange, they also pose a number of significant challenges. One major challenge is determining whether news on social media is real or fake, especially now that we are more aware of fake news (Marchal et al. 2024). Fake news is misleading information aimed at swaying public opinion or disparaging an individual. Fake news includes rumors, opinions, conspiracy theories, spam, parody news, misinformation, disinformation, Clickbait, and fake reviews, to name a few (Kumar et al. 2024, 2023). Fake news has been shown to get more attention on social media than real news, with 10 fake news items on the popular social networking site "Facebook" getting more user interaction than the top 10 real news items. Social media features like posting, commenting, sharing, and tagging friends in a post have been noticed as being relatively easy for spreading these stories (Pandey et al. 2022). Therefore, it is crucial to prevent the spread of both fake news. To address this, we developed a novel solution to identify fake news at an early stage. As an example, we discussed a fake news story regarding the Padma Bridge in Bangladesh.

The Padma Bridge is a significant milestone for Bangladesh. However, when construction began, a disturbing piece of fake news spread on Facebook, claiming that building the first pillar required the severed heads of children. It was falsely reported that kidnappers had been hired to abduct children and collect their heads. One day, a mother named Renu, appearing disheveled, picked up her daughter from school, and the child cried and pleaded for something. At that moment, another guardian named Riya shouted, "Child lifter!" which led to the surrounding crowd brutally beating Renu to death in front of her daughter (Amin 2020). This story motivates us to develop new solutions for identifying false news at an early stage.

The spread of misleading information through media channels such as online newspapers and social media posts has underscored the need also to identify trustworthy news sources (Udoy et al. 2023; Allen et al. 2024). These digital media channels have significantly influenced our daily routines and online behaviors. Consequently, we must exercise great caution when posting and sharing content on social media. Users are responsible for assessing the credibility of information and filtering out fake news, as it is often widely disseminated through digital platforms. Now, we discuss how to address the issue of false news and review the existing work on this topic. Several methods have been attempted to address this issue, with the most popular approach being machine learning combined with NLP to identify and stop the spread of fake news. Machine learning has broad applications, including computer vision, face and speech recognition, image analysis, human-computer interaction, fraud detection, robotics, cybersecurity (Shaha et al. 2024), loan credit approval, and bank failure prediction, among others. These methods have shown promise as effective tools (Lahby et al. 2022). Many researchers have utilized machine learning and deep learning in different ways to identify fake news. For example, Madani et al. (2024) proposed a hybrid technique combining a curriculum strategy with KNN to enhance deep learning performance, suggesting a two-phase model using NLP and ML algorithms to extract new structural features from news samples. Abualigah et al. (2024) applied DL methods such as LSTM, DNN, and CNN for fake news detection. Other works (Dev et al. 2024; Rabbi et al. 2025; Mallik and Kumar 2024; Mahmud et al. 2024; Mahir et al. 2025) used LSTM, Word2Vec, and TF-IDF with ML and DL algorithms for false news identification, while Ghamdi et al. (2024) leveraged BERT and NLP along with ML to identify fake news. Further studies have explored advanced deep learning architectures, such as the use of RoBERT in Kumar et al. (2022) for fake news detection, and the application of multi-head attention dual summarization in Kumar et al. (2022) to detect incongruent news articles. The aforementioned research has focused on identifying fake news using various techniques, yet some limitations need further attention. To tackle these challenges, we proposed a machine learning approach integrated with natural language processing for detecting fake news. This research examines how ML can effectively identify false news. The key contributions of our study are outlined below:

- We proposed an enhanced feature vectorization-based model for detecting fake news in both Bangla and English. This study employs Term Frequency-Inverse Document Frequency (TF-IDF) for feature extraction. Additionally, the Synthetic Minority Over-Sampling Technique (SMOTE) is applied to balance the Bangla dataset.

- We applied several machine learning algorithms, including SVM, DT, RF, LR, KNN, GNB, XGB, and LightXGB, along with AdaBoost, Voting Classifier, and the Stacked model. The Stacked model, an ensemble learning technique, improves classification accuracy by combining the strengths of multiple algorithms. In this model, we used five distinct machine learning algorithms: SVM, RF, LR, GB, and XGBoost.
- The Stacked model is our proposed approach, and its performance is evaluated using Accuracy, Precision, Recall, F1-score, and the ROC-AUC Curve. The source code is available in the below link: https://github.com/PabonSC/Fake-News-Detection.

The following headings outline the remaining sections of our study: Sect. 2. This chapter summarizes the research papers and surveys currently available on recognizing fake news using techniques. Section 3 This section explains the required specifications, associated diagrams, and system architecture of our approach. Section 4. In this chapter, we detail the machine learning models used, supported by information, tables, graphs, charts, and visuals of the tools employed. Section 5 This section summarizes the key findings to conclude the study and offers insights into potential future applications of the research.

2 Literature review

Fake news is a common problem worldwide. Several studies, including the use of ML and DL, have been conducted to eliminate or stop the spread of false news. This section mentions a few of them that have been proposed before to detect fake news.

2.1 Fake news detection techniques

There are both conventional and contemporary methods for detecting fake news. Rule-based systems evaluate language patterns, like keyword frequency or sentence structure, to detect potentially erroneous information without the use of ML or DL. Cross-referencing claims with credible sources and fact-checking websites are other popular techniques. According to their author or place of origin, suspicious articles can be identified using methods such as content-based filtering and metadata analysis. When contrasted with ML and DL-based techniques, these strategies are less flexible and scalable.

2.2 Fake news detection through NLP and ML

To detect fake news, Asha et al. (2024) utilized machine learning algorithms RF, NB, and LR to detect various types of false information and achieve accurate news classifications. Among these, the RF algorithm demonstrated superior performance, achieving an experimental accuracy of 99.06%, surpassing both Logistic Regression and Naive Bayes. The Random Forest approach consistently produced better results. Holla and Kavitha (2024) presented an improved model to identify fake news that combines the Iterative Dichotomiser 3 (ID-3), NB, and RF classifiers with the Adaptive boosting ensemble classifier. The proposed Hybrid TF-IDF was utilized to extract features. The AdaBoost classifier classifies the news as true or false, and the characteristics are chosen using the Least Absolute Shrinkage and Selection Operator (LASSO). The findings demonstrated an increased classification accuracy of 98.98% for TF-IDF using the AdaBoost ensemble classifier. However, the proposed model was not designed to identify false news based on real-time applications. Choudhury and Acharjee (2023) compared the effectiveness of SVM, NB, RF, and LR classifiers for identifying false news across several datasets. Fake News datasets, Fake Job Posting, The Liar showed that the SVM reached the greatest accuracy at 96%, 97%, and 61%, respectively. Again, their innovative GA-based false news detection system considers the fitness functions of SVM, NB, RF, and LR. The LIAR dataset yielded 61% accuracy for both the SVM and LR classifiers in their suggested approach, while the fake job posting dataset produced the highest accuracy of 97% for SVM and RF. Altheneyan and Alhadlaq (2023) used machine learning and Big data technology (Spark) to assess and compare the most sophisticated techniques for identifying fake news. This study's methodology used a decentralized Spark cluster to build stacked ensemble models. After extracting features with a count vectorizer, TF-IDF hashing, and N-grams, the authors applied the suggested stacked ensemble classification model. According to the findings, the Hasing TF-IDF ensemble achieved 93.45% of the highest accuracy. Also, the suggested approach performs better at classifying data than the baseline strategy, with an F1 score of 92.45% compared to 83.10%. Jain et al. (2024) introduced a brand-new "ConFake" algorithm. This algorithm includes eighty content-based features to help users to spot fake news. The project made use of word vectors and content-based attributes that were taken directly from the text of news articles. Combining these traits, they were fed into machine learning classifiers. They conducted all of the tests on five publicly accessible datasets and one artificially created ConFake dataset that combines multiple datasets, including BuzzFeed, PolitiFact, Reuters, Kaggle, and McIntire, to

validate the experimental results. Comparing the suggested model to other cutting-edge models, it obtained a maximum accuracy of 97.31%. However, finding false news on social media early on is not a suitable use of this model, as the text field of news stories is the only aspect the model examines. Asha and Meenakowshalya (2021) utilized n-gram analysis and ML methods to identify false news. This study used six machine learning techniques: KNN, DT, SVM, LSVM, LR, and stochastic gradient descent. With a 93.5% accuracy rate, the study reveals that the highest result is achieved while using the feature extraction approach TF-IDF, together with LSVM and Stochastic Gradient Descent algorithms. Performance tuning is used in Stochastic Gradient Descent to improve accuracy. When Random Search CV and Grid Search CV are compared, the latter provides more accuracy 94.2%.

Mugdha et al. (2020) proposed an approach that can precisely ascertain whether a story is real or not based on the headline of the news. The authors used the Gaussian NB to produce a unique dataset of the Bengali language, achieving their goal and reaching their target. Although they have experimented with several Machine Learning methods, their model has shown that the Gaussian NB Algorithm worked well. To choose the attribute, this technique used an Extra Tree Classifier in conjunction with utilizing a text feature that was dependent on TF-IDF. They achieved 87% accuracy using Gaussian NB, which was superior to any other technique they employed. Hussain et al. (2020) presented experimental research to determine false news on Bangla social media. They used two supervised ML techniques, SVM and Multinomial NB, to identify false news in Bangla. Count Vectorizer and TF-IDF were employed as feature vectorization methods. Their proposed method identifies false news based on the connected post's polarity. Ultimately, their research indicated that SVM with a linear kernel outperforms Multinomial Naïve Bayes with a 93.32% accuracy, yielding 96.64% accuracy. Coelho et al. (2023) utilized machine learning algorithms to identify false news in Malayalam languages. TF-IDF of word unigrams is used to train three distinct models to detect false news in code-mixed Malayalam language. These models are Multinomial Naive Bayes, LR, and an Ensemble method (SVM, LR, and MNB) with hard voting. The ensemble model performed the best out of the three, with a macro accuracy of 83.1%. Farooq et al. (2023) suggested a methodology for identifying false news in Urdu. A bag of words containing n-gram combinations and the TF-IDF is used in the experiments. The authors used feature stacking, a technique that mixes verbs extracted from the text with the feature vectors of preprocessed text. Bagging was done using SVM, KNN, and ensemble models such as RF and ExtraTreeClassifier, while stacking was done using RF and ET as base learners and LR as the meta learner. Stacking yields the maximum accuracy of 93.82%, according to experimental data.

2.3 Fake news detection through NLP and DL

Balshetwar and Rs (2023) suggested a fake news identification model that uses Multiple Imputation Chain Equation (MICE) with multiple imputations to handle multivariate missing data in news or social media datasets. TF-IDF was employed to extract long-term features from the text. The classification was performed using passive-aggressive, Deep Neural Network (DNN), and NB classifiers, achieving a 99.8% accuracy rate in identifying false news Pal and Pradhan (2023) introduced a model that aids in identifying false news while dealing with data gathered from the internet. These articles served as their training dataset, and they compared the outcomes using a variety of ML algorithms, including NB, LR, DT, LSTM, and BERT. In their experiment, they were able to identify false news with 99.6% accuracy using the DT and 99.8% recall using the LSTM. Alghamdi et al. (2023) introduced a model to detect false news related to COVID-19; this study investigated the efficacy of several ML method and the refinement of pre-trained transformer-based models, such as COVID-Twitter-BERT (CT-BERT) and BERT. The authors assess the effectiveness of several downstream neural network architectures, with frozen or unfrozen parameters, added to BERT and CT-BERT, such as BiGRU and CNN layers. With an accuracy of 98.46%, the COVID-19 false news dataset shows that adding BiGRU to the CT-BERT model produces exceptional results. Salh and Nabi (2023) suggested a model to identify false news in Kurdish language. Word embedding, TF-IDF, and count vector were utilized to extract features from news texts. RF, SVM, and other ML and DL classifiers were used to identify the false news. The findings demonstrated that it was possible to spot fake news that contained text, particularly when convolutional neural networks were applied. The experimental findings of the study demonstrate that CNN outperforms the other methods, with an accuracy of more than 91%. Azzeh et al. (2025) constructs a large annotated Arabic fake news corpus and develops machine learning models using Arabic-specific language models (ARBERT, AraBERT, CAMeLBERT, and AraVec), finding that deep learning classifiers with transformers, especially CAMeL-BERT, outperform traditional machine learning approaches. How ever detecting fake news in Arabic presents unique challenges, including a lack of annotated data, dialectal variations, morphological complexity, semantic ambiguity, and cultural context. Khalil et al. (2025) introduces Pegasos Quantum Support Vector Machines (PegasosQSVM), combining Pegasos SVM with quantum kernels and advanced data encoding, achieving up to 95.63% accuracy in local

simulations and outperforming other ML models on the BUZZFEED dataset. While the approach shows promise, implementation on real Quantum Processing Units (QPUs) faces challenges due to noise effects, necessitating further research on error mitigation techniques for optimal performance. Table 1 shows the summary of existing works on Fake News Detection.

In summary, researchers have frequently employed various machine learning (ML) and deep learning (DL) algorithms alongside natural language processing (NLP) techniques to detect fake news in English, Arabic, and other languages. However, limited research has been conducted on the low-resource language Bangla. Moreover, existing approaches suffer from several key limitations: researchers focus on single-language fake news detection without experimentation in other languages, many are unsuitable for detecting fake news on social media, some exhibit lower accuracy, others lack feature extraction techniques, and several are not designed for real-time applications. To address these challenges, we introduced a novel machine learning model for multilingual fake news detection, leveraging advanced feature extraction techniques. Our proposed model combines feature extraction methods, including TF-IDF, mBERT, and Word2Vec, with eleven traditional and state-of-the-art classifiers to enhance classification performance. Designed to work across multiple languages, particularly low-resource languages like Bangla. The model was first tested on an English fake news detection dataset, achieving higher accuracy than previous studies. Additionally, we applied the model to Bangla fake news detection on a dataset with no prior research and obtained satisfactory accuracy rates, demonstrating its effectiveness across diverse linguistic contexts. We further evaluated the model on a second Bangla dataset, BanFakeNews, where it continued to perform strongly, confirming its robustness across multiple Bangla datasets.

Table 1 Summary table for related works on fake news detection

Refs.	Algorithms/technique	Major findings	Backdrop
Asha et al. (2024)	RF, NB, and LR	Random forest achieved 99.06% highest accuracy	No feature extraction technique used
Holla and Kavitha (2024)	ID-3, NB, RF classifiers with Adaptive Boosting	AdaBoost achieved 99.06% highest accuracy	Not designed for real-time application
Choudhury and Acharjee (2023)	SVM, NB, RF, and LR	SVM achieved 97% highest accuracy	Not suitable for fake news detection from social media
Altheneyan and Alhadlaq (2023)	RF, DT, LR, Ensemble	HashingTF-IDF_Ensembler achieved 93.45%	Accuracy is comparatively lower than others
Jain et al. (2024)	SVM, RF, KNN, NB, LR, Bagging, Boosting	Random Forest achieved 97.31% accuracy	Model examines only the text of news articles
Asha and Meenakowshalya (2021)	DT, LR, KNN, SVM, LSLVM, SGD	Linear SVM achieved 93.5% accuracy	Not suitable for social media fake news detection
Mugdha et al. (2020)	Gaussian NB, SVM, LR, RF, Multinomial NB, AdaBoost, GB, Voting Classifier	Gaussian NB achieved 87% accuracy	Accuracy is quite lower than others

3 Materials and methods

In this study, we aimed to train a model that can accurately distinguish between true and fake news, with the goal of benefiting our community. By detecting fake news early, we sought to limit its dissemination and prevent further spread. Figure 1 presents the proposed fake news identification model. This section outlines the step-by-step methodology used to develop the model. The process began by gathering datasets. After collecting the data, preprocessing steps were performed to prepare it for machine learning classifiers and feature extraction. Data preprocessing included handling null values, tokenization, stemming, lemmatization, and stop-word removal. Additionally, we employed SMOTE to balance the Bangla dataset (dataset-2). mBERT, Word2Vec, and TF-IDF vectorizer were utilized for feature extraction. After preprocessing, the extracted features were fed into various classifiers to predict fake news. The classifiers used included SVM, RF, DT, LR, GB, NB, XGB, AdaBoost, LightGBM, Voting, and Stacking models. The models' performance was assessed after training. using confusion matrices, and performance metrics such as accuracy, recall, F1-score, and precision were utilized. The procedures and models employed in this study are detailed in the following sections. However, certain procedures were excluded due to the inclusion of the Bangla language alongside English, as some tools required for Bangla were unavailable.

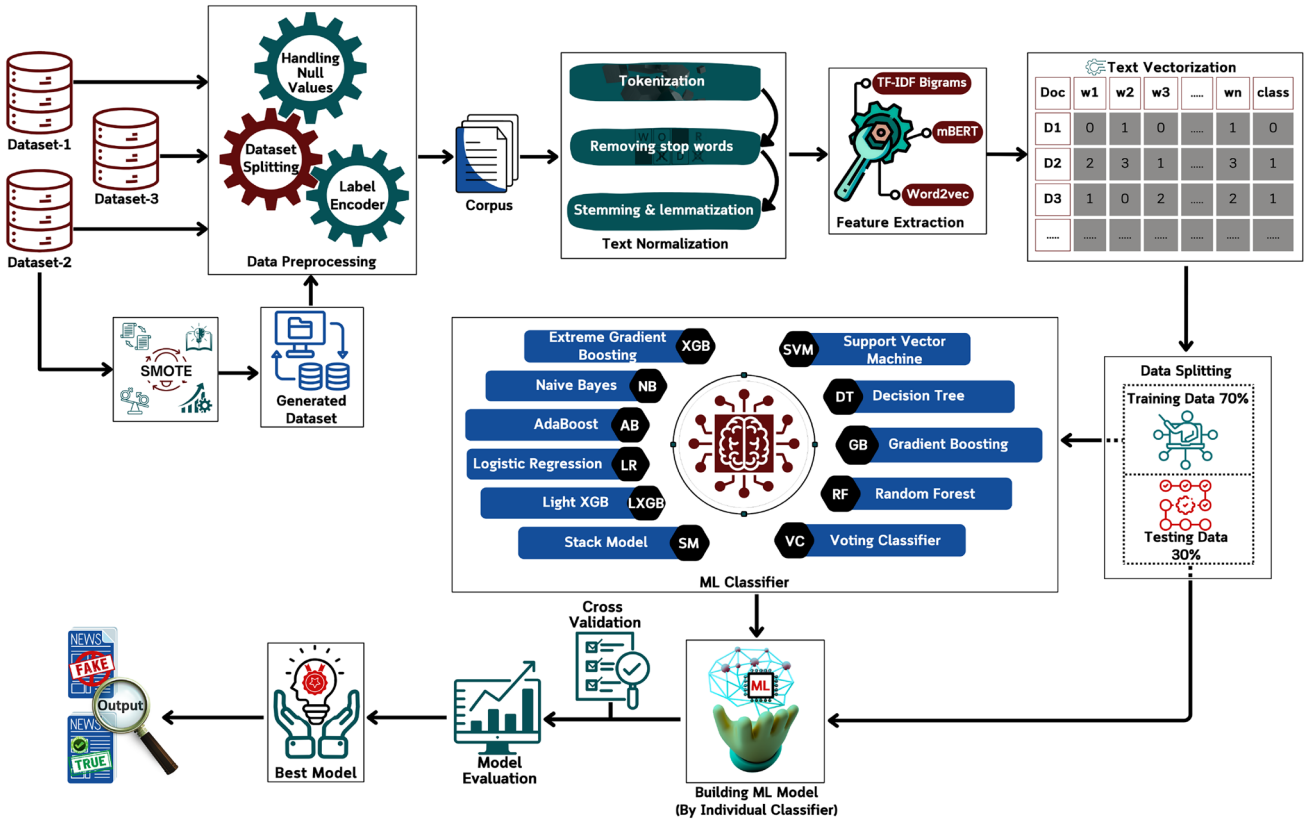


Fig. 1 The overall framework for the fake news detection system

Table 2 Dataset description

Data set no.	Total data	Real news	False news
Dataset 1	10,000	5000	5000
Dataset 2	14,537	10,000	4537
Dataset 3	9816	7212	2604

3.1 Dataset description

We used three datasets to distinguish between true and false news. Two of them were sourced from Kaggle: the Bangla and English Fake News Detection Datasets, comprising two separate CSV files *final_en.csv* for English fake news and *final_bn_data.csv* for Bangla fake news. The third dataset, titled Banfakenews, was collected from a published research article (Hossain et al. 2020). Together, these datasets provide a total of 34,353 data points. For the purposes of this study, we refer to the English Kaggle dataset as *Dataset-1*, the Bangla Kaggle dataset as *Dataset-2*, and the Banfakenews dataset as *Dataset-3*. Table 2 provides an overview of the dataset's properties used in the study.

3.1.1 English dataset

English is one of the most extensively used languages in the world, and it is used to write data in the English dataset. The English dataset is structured in CSV format, utilizing a tabular layout of rows and columns. It contains 10,000 entries and 3 features, where each row represents a data point. The dataset is evenly balanced, with 5,000 entries classified as real (1) and 5,000 entries classified as false (0).

3.1.2 Bangla dataset

The Bangla dataset consists of information written in Bengali, which is primarily spoken in Bangladesh and some regions of India. It is in CSV file format, a popular row-and-column tabular data format. The dataset comprises data or samples that have been determined as real (1) or false (0). The Bangla dataset has 14,537 records and 4 features. In this case, 4537 news outlets are false, and 10,000 news outlets are real.

3.1.3 BanFakeNews dataset

The Banfakenews dataset comprises news content written in the Bangla language, aimed at detecting misinformation in Bangladeshi media sources. It is provided in CSV file format, organized in a structured tabular form. The dataset includes a total of 9816 entries, where each row represents a labeled news article. Of the total samples, 7212 entries are

labeled as real (1), while 2604 entries are labeled as fake (0) news.

To visualize the data distribution, we employed pie charts. Figure 2 illustrates the distribution of real and fake news categories for the label classes.

3.2 Data pre-processing strategies

The dataset, being in natural language, inherently contains a lot of noise. To prepare the data for use with algorithms, it undergoes several preprocessing steps. Data preprocessing is the process of repeatedly converting raw, unprocessed data into a more useful and understandable format. Raw datasets often have issues such as errors, missing values, inconsistencies, and lack of clear patterns or behavior. Preprocessing is crucial for addressing these problems by handling missing values and reconciling inconsistencies. To improve efficiency, preprocessing modifies the data before it is processed by algorithms, ensuring it is in a suitable form for further analysis.

3.2.1 Balancing the dataset using the synthetic minority oversampling technique (SMOTE)

The Bangla dataset originally consists of 14,537 data points, with 10,000 real news items and 4537 fake news items, resulting in a class imbalance. To address this, we applied the Synthetic Minority Over-sampling Technique (SMOTE), which interpolates between samples of the minority class and their nearest neighbors to generate synthetic data points. By adding 5463 synthetic samples to the fake news category, the dataset was expanded to 20,000 samples, achieving a balanced distribution of 10,000 true and 10,000 fake news. This process reduced bias towards the dominant class and improved model performance by providing a more equitable representation of both classes.

3.2.2 Handling null values

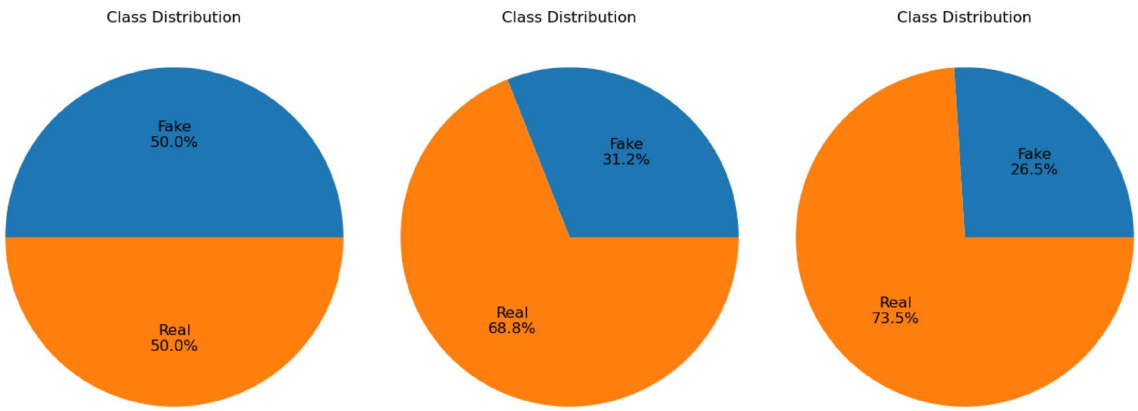
Handling null values is essential for maintaining data quality and improving model performance. Missing values can be addressed through deletion or imputation using methods like mean, median, or mode. Proper handling reduces errors and enhances model accuracy, ensuring a consistent and reliable dataset for training. For text data, missing values can be replaced with an empty string or imputed based on context. However, in the fake news dataset, no null values were found.

3.2.3 Label encoding

Label Encoding is a method used to convert categorical labels into numerical values. It helps machine learning models process categorical data efficiently. In the fake news dataset with two classes (Real and False), Label Encoding assigns *0 to Real* and *1 to False*. This transformation ensures the model can interpret and analyze the labels effectively. It also preserves the categorical relationships within the dataset.

3.2.4 Splitting the dataset

To split the dataset, the K-fold cross-validation technique was used in our proposed method. K-fold Cross-Validation aims to divide the dataset into k segments. Each iteration uses $(k-1)$ segments for training and the K^{th} segment for testing. This ensures that the entire dataset is used for both training and testing. Cross-validation has been employed to assess our performance. Here, $k = 5$ was employed for the procedure. We receive 30% of the data for testing and 70% for training in each cycle due to splitting the dataset into five folds. The dataset is labeled as (1) and (0), where (1) represents real news and (0) represents fake news. The system



(a) Dataset-1 (b) Dataset-2 (c) Dataset-3

Fig. 2 Class distribution of three datasets, visualizing using Pie charts

distinguishes between real and false, determines whether the news is accurate or inaccurate, and computes the accuracy of the models as a percentage based on the ratio of correct to incorrect predictions.

3.3 Text normalization

Text normalization is a necessary step in natural language processing (NLP). To make text data consistent and useful for various NLP tasks, it must be cleaned and preprocessed. Many approaches were used in this procedure, including,

3.3.1 Tokenization

The method of encoding every word as a number is called tokenization. Within the fields of Machine Learning and NLP, the tokenization procedure for a text sequence entails partitioning it into smaller pieces called tokens. The length of these tokens might vary from word to character. Tokenization is the process of parsing text to eliminate particular words to use for predictive modeling.

3.3.2 Stop Words

The words 'a,' 'an,' 'the,' 'are,' and 'is' are a few instances. In NLP and text mining, stop words are frequently used to filter out overused words that don't provide any valuable information. In this case, the stop word dictionary is provided by NLTK. First, every stop word in the text is eliminated to tidy it up. Stop words can be eliminated from the text because they are less informative and more common. Stop words that are frequently used are the conjunctions "and," "or," and "but." Because processing these less common whole words takes a long time, pre-processing data is crucial to natural language processing.

3.3.3 Stemming and lemmatization

This phase involves either stemming or lemmatization. For lemmatization, the NLTK uses its WordNet Lemmatizer; for stemming, it uses its Snowball Stemmer implementation, which is based on the Porter2 stemming method.

- *Stemming:* To simplify an inflected word to its word stem, employ the stemming process. The common word stem "Detect" can be used to represent the terms "Detection," "Detector," and "Detects," for instance. In other words, the three inflection words before "Detect" can be substituted with "Detect."
- *Lemmatization* Another method for returning inflected words to their source word is lemmatization. It explains the algorithmic procedure for determining the dictionary form, or "lemma," of an inflected word by considering its intended meaning. To lemmatize a document usually means "doing things correctly." A lemmatization algorithm, for instance, should convert the terms "identify," "identifying," and "identification" into the lemma "identify."

3.4 Feature extraction

The technique of feature extraction converts unprocessed data into numerical characteristics that may be used for additional processing while maintaining the information contained in the original data set. When compared to just training a machine with raw data, it is more efficient. Word2Vec, mBERT, and TF-IDF are the three categories of methods for feature extraction techniques we utilized in the proposed model.

3.4.1 TF-IDF

TF-IDF or Term Frequency-Inverse Document Frequency, is a machine-learning metric that quantifies the significance or pertinence of string representations (words, phrases, lemmas, etc.). Term Frequency or TF is a tool for determining a term's frequency of occurrence in a document. Given that all documents have different sizes, A term can occur more often in a longer text than in a shorter one. Frequently, TF is divided by the length of the document. TF is used for text summarization, search engine optimization, and document clustering (Ramos 2003).

$$\mathrm{TF}(t,d) = \frac{\text{Number of times term } t \text{ occurs in document } d}{\text{Total word count of document } d} \tag{1}$$

IDF, or Inverse Document Frequency, states that a word has little value if it appears in every document. A text may contain numerous instances of words that are not very important, such as "a," "an," "the," "on," and "of." IDF gives uncommon terms more weight while lowering the value of familiar terms. The word's IDF value increases its uniqueness (Sharma et al. 2020).

$$\mathrm{IDF}(t,d) = \frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}} \tag{2}$$

When TF-IDF is utilized in the text body, each sentence's relative count is noted in the document matrix. Sparse matrices are produced using vectorizers. A sparse matrix is one in which the majority of the elements are 0. The following is the equation for TF-IDF.

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t) \tag{3}$$

3.4.2 Word2Vec

The Word2Vec model functions based on the core idea that a word's meaning is captured by its context. This model, in short, creates each word as a multidimensional vector representation. At the same time, it maintains the syntactic and semantic connections between them. Word associations as seen by humans are reflected in the vector distances between individual words (Muhammad et al. 2021). The two main designs of the Word2Vec model are Continuous Bag-of-Words (CBOW) and Skip-gram. Whereas, within a specific window, CBOW forecasts a target word determined by the enclosing context words (Kapusta et al. 2024). The goal of skip-gram is to anticipate context words within a particular context window, given a target word. These structures can be thought of as simple neural networks having input, hidden, and output layers. The hidden layer's weights, which function as word vectors, are the training results of these networks rather than the output layer itself. To minimize computational complexity, Word2Vec uses algorithmic improvements, including hierarchical SoftMax and negative sampling, in addition to using a SoftMax activation operation for the output layer (Mikolov et al. 2013).

3.4.3 mBERT

Multilingual BERT (mBERT) is a transformer-based language representation model that provides contextual embeddings for over 100 languages, including low-resource languages like Bangla. Unlike traditional word vectorization techniques such as TF-IDF or Word2Vec, which are based on frequency or co-occurrence statistics, mBERT leverages deep bidirectional transformers to understand semantic meaning in context. It was pretrained on a large multilingual corpus using masked language modeling and next sentence prediction objectives, allowing it to learn syntactic and semantic relationships across different languages (Devlin 2018).

mBERT processes each sentence by first converting it into tokens and then generating dense vector representations through its multiple attention-based layers. Typically, the contextualized embedding of the special classification token [CLS] is extracted and used to represent the entire sentence in a fixed-size vector of 768 dimensions. Alternatively, the embeddings of all tokens can be averaged or pooled for sentence-level representation. This contextual embedding captures both local and global linguistic information and is robust to polysemy and word order variations (Hossen et al. 2025). In this study, mBERT was employed to generate dense contextual embeddings for each input text. These embeddings, which capture semantic and syntactic information across languages, were used as feature vectors for downstream classification. The resulting vectors were then fed into eleven different machine learning classifiers to evaluate their performance on the classification task.

3.5 Data visualization

Information and data are represented graphically in data visualization. It uses visual elements such as charts, graphs, maps, and other tools to present data, making complex datasets more straightforward to understand, analyze, and interpret. In this research, we used the following techniques.

3.5.1 Histogram and box plot

A histogram is a graphical representation of the distribution of data. It divides the range of data into intervals, called bins, and counts how many data points fall into each bin. This helps visualize the frequency distribution of the dataset.

A box plot summarizes the distribution of a dataset by displaying its key descriptive statistics, including median, quartiles, and outliers.

We present two data visualization approaches for the original dataset. Figure 3(a) shows the histogram, and (b) depicts the box plot for Dataset-1 before applying TF-IDF.

Figure 4(a) presents the histogram, while (b) illustrates the box plot for Dataset-2 before applying TF-IDF. The histogram indicates that Dataset-2 is imbalanced. To address this, we applied SMOTE to balance both classes in Dataset-2.

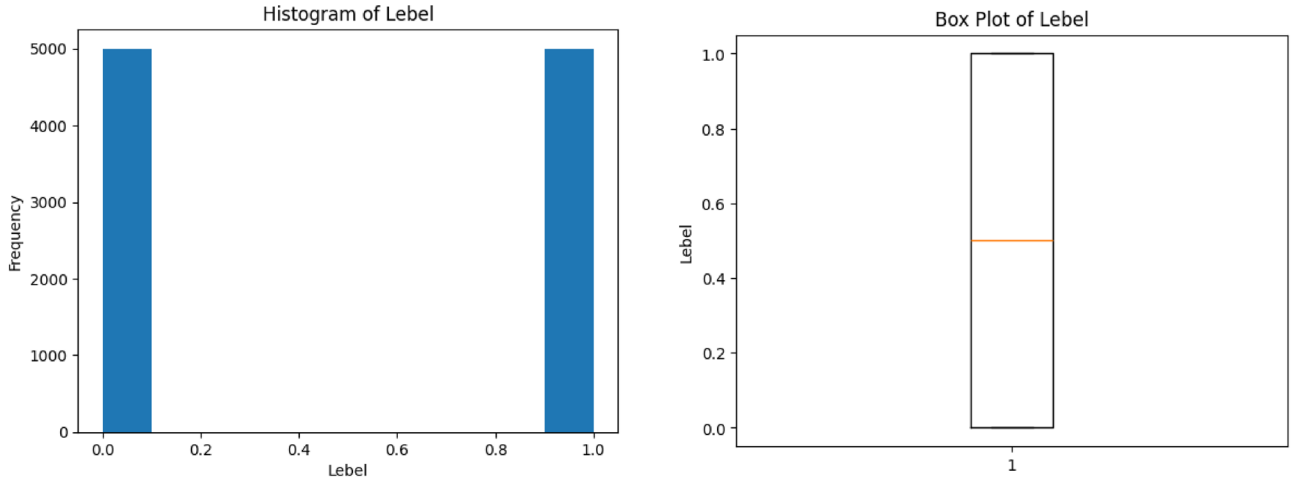
Figure 5(a) shows the histogram, and (b) depicts the box plot for Dataset-3 before applying TF-IDF.

3.5.2 UMAP and t-SNE

We applied two feature vectorization techniques, TF-IDF and Word2Vec, to both datasets. TF-IDF consistently demonstrated superior performance. Therefore, we present t-SNE and UMAP data visualizations applied exclusively to the TF-IDF vectorizer.

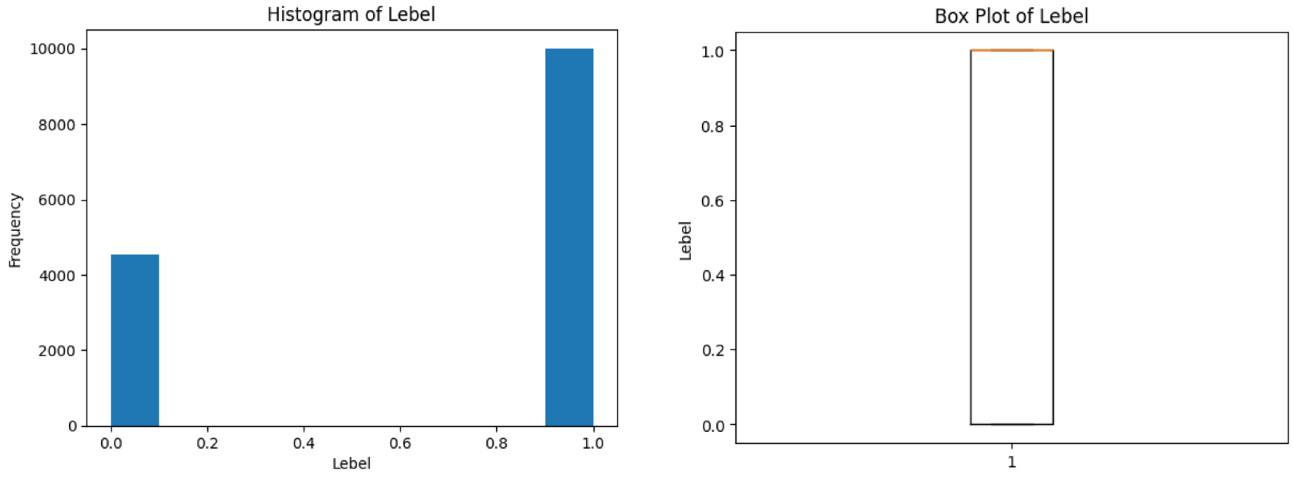
t-Distributed Stochastic Neighbor Embedding, or t-SNE, is a dimensionality reduction technique that preserves local structure while visualizing high-dimensional data by mapping it onto a low-dimensional space. Although it is frequently used for data clustering and pattern recognition, it can be computationally demanding for large datasets.

An alternative to t-SNE that is quicker and preserves both local and global structures while reducing dimensionality is UMAP (Uniform Manifold Approximation and Projection). It can effectively visualize complex data distributions and is scalable.



(a) Histogram (b) Box Plot

Fig. 3 Histogram and box plot representation for Dataset-1



(a) Histogram (b) Box Plot

Fig. 4 Histogram and box plot representation for Dataset-2

Figure 6(a) shows the t-SNE visualization, while (b) presents the UMAP visualization of Dataset-1 after applying TF-IDF.

Figure 7(a) illustrates the t-SNE visualization, while (b) depicts the UMAP visualization of Dataset-2 after applying TF-IDF.

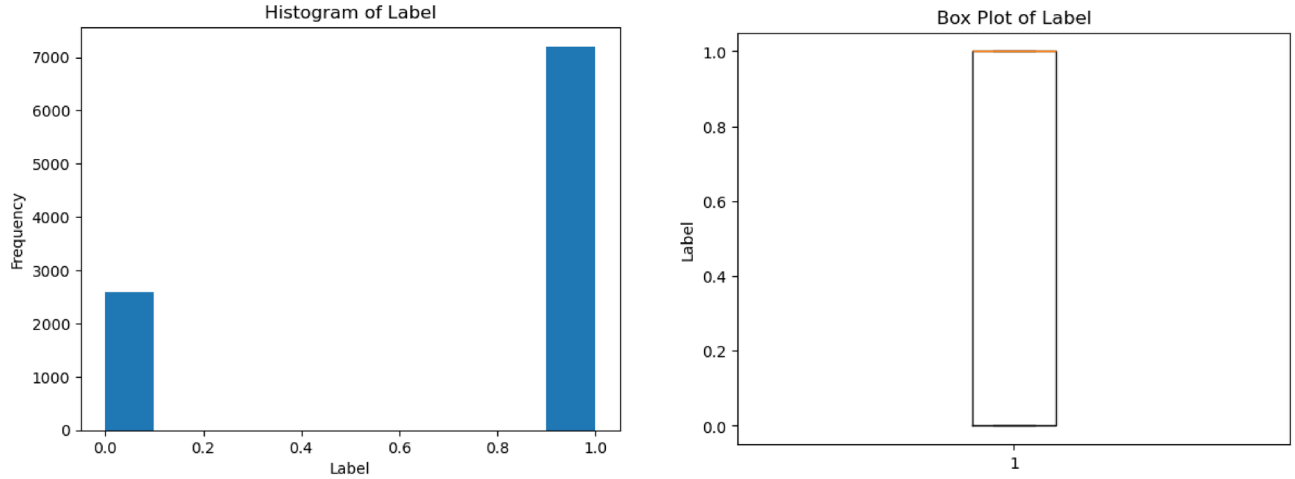
Figure 8(a) displays the t-SNE visualization, and (b) presents the UMAP visualization of Dataset-3 after applying TF-IDF.

3.6 Analysis of classification procedures

The multifaceted structure of fake news makes it difficult to identify which genre of news it belongs to. It is clear that to address the problem precisely, a practical technique needs to incorporate several views. For this reason, several Machine Learning algorithms were employed as classifiers, such as SVM, RF, DT, LR, GB, NB, XGB, AdaBoost, LightGBM, Voting, and Stacking models. Details of these models are as follows.

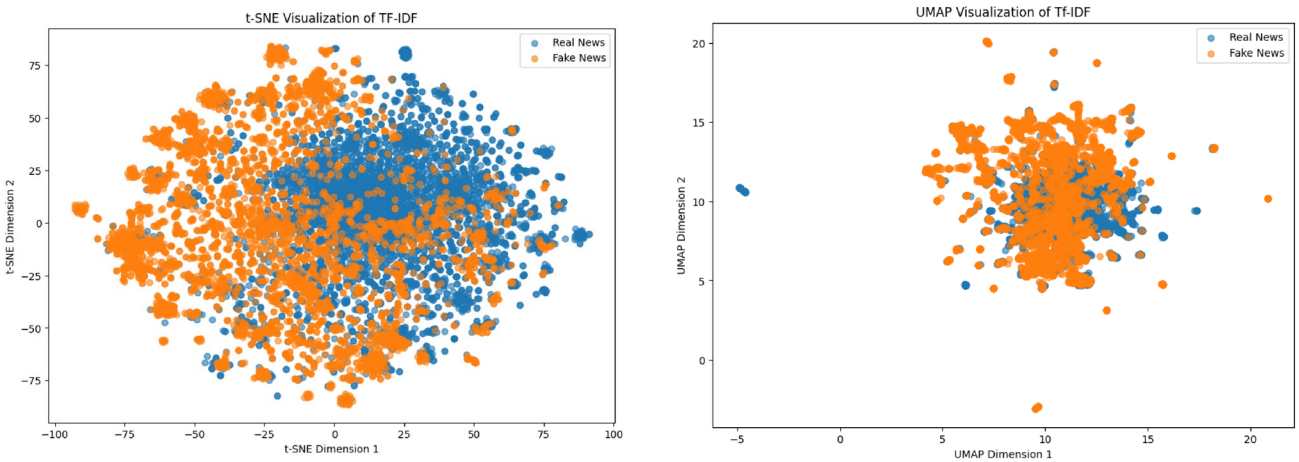
3.6.1 Support vector machine—SVM

SVM is a supervised learning technique effective for both regression and classification tasks, particularly binary classification. It uses labeled data to learn and determine the optimal hyper-plane that separates the dataset into two classes: true and false (Suthaharan 2016). The decision boundaries, known as hyperplanes, help classify the data



(a) Histogram (b) Box Plot

Fig. 5 Histogram and box plot representation for Dataset-3



(a) t-SNE (b) UMAP

Fig. 6 t-SNE and UMAP for Dataset-1 after apply TF-IDF vectorization

points. Figure 9 illustrates how SVM uses a hyper-plane for data point classification.

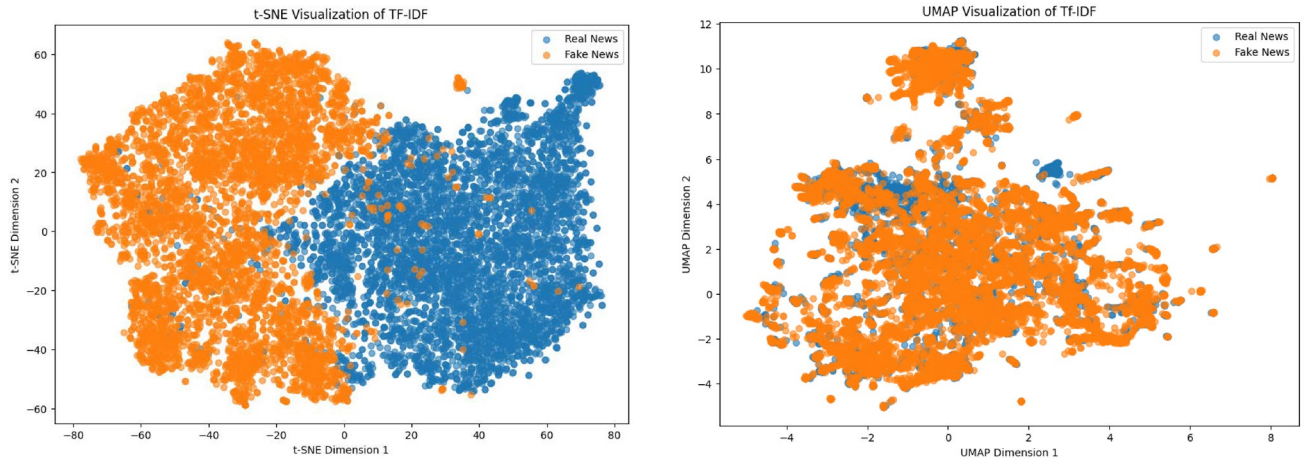
In this study, the SVM algorithm generates hyper-planes based on feature vectors obtained through TF-IDF, Word2Vec,and mBERT, which capture the importance and semantic relationships of words. These vectors are used to construct hyperplanes that optimally separate the data into two classes: fake and real news. The SVM then classifies new articles by determining on which side of the hyperplane their feature vectors fall, accurately predicting whether the news is fake or real.

3.6.2 Random forest—RF

One kind of supervised learning technique that works well for both regression and classification is called RF (Adib et al. 2021). The RF is utilized in this study to identify whether the news is genuine or fake. Additionally, an RF is made up of decision trees on a sample of data, and the best solution is chosen by voting. The overfitting issues can be resolved, and the RF classifier improves detection accuracy (Breiman 2001; Alhussan et al. 2022). The following is a mathematical expression of the procedure used to train the data in RF found in Eq. (4):

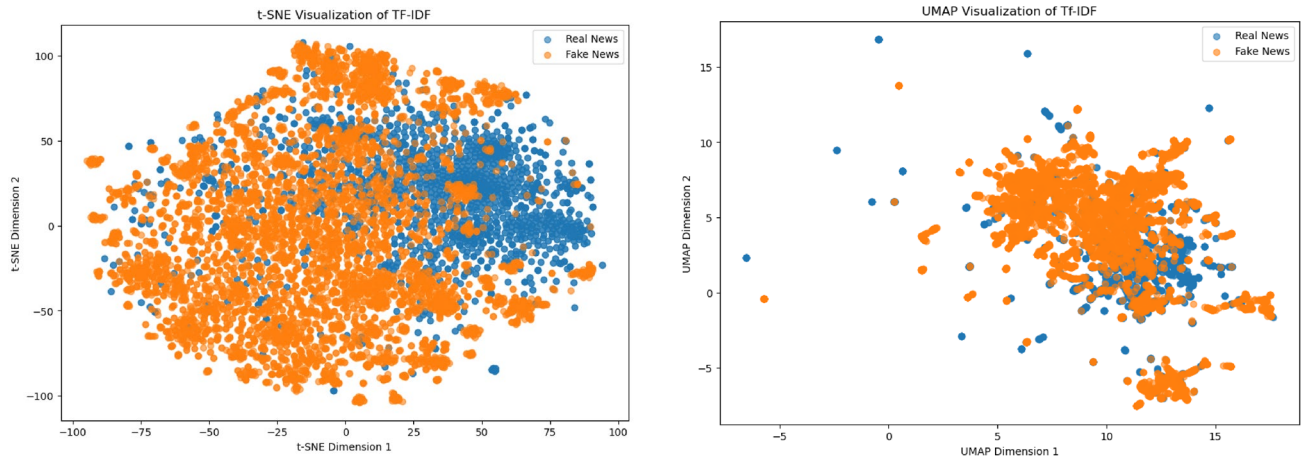
$$D = \left\{ (f_i, C_i) \mid f_i \in \mathbb{R}^F, C_i \in \{1, 2, \ldots, c\} \right\}_{i=1}^{N} \tag{4}$$

Where N denotes the number of samples used for the training, f_i represents the features, and C_i represents the count of samples used for training. Each Decision Tree (DT) in the Random Forest (RF) classifier consists of a bag of samples



(a) t-SNE (b) UMAP

Fig. 7 t-SNE and UMAP for Dataset-2 after apply TF-IDF vectorization



(a) t-SNE (b) UMAP

Fig. 8 t-SNE and UMAP for Dataset-3 after apply TF-IDF vectorization

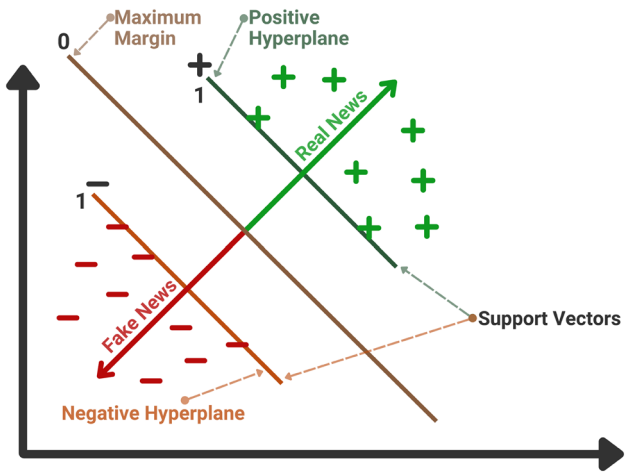


Fig. 9 Hyperplane illustration for support vector machine

(i.e., $D_1, D_2, \ldots, D_P$, etc.). The following formula, Equation (5), is used to determine the detection accuracy:

$$\text{D.A} = \frac{\text{Number of Correctly Classified Instances}}{\text{Total Number of Instances}} \times 100 \tag{5}$$

"D.A., which stands for Detection Accuracy."

$$\hat{C} = \text{majority}\left\{\hat{C}^d\right\}_1^d \tag{6}$$

In this study, numerous decision trees are generated based on feature vectors. A majority vote from these decision trees determines the final classification of the news as fake or real.

3.6.3 Decision tree—DT

DT is a popular ML technique and a non-parametric supervised learning approach in both regression and classification. Using resource costs, chance events, and utility, a decision tree is a decision assistance tool that displays options and their potential results in a hierarchical model (Song and Ying 2015). It has an internal, leaf, branches, and a root node, arranged like a hierarchical tree. Decision trees are used to construct models for tasks involving regression and classification, and are simple to comprehend and interpret, making them ideal for visual aids (Kaur et al. 2020). In this research, we employed the Decision Tree algorithm by following the steps:

- Decide on the desired attribute.
- Determine the Information Gain (I.G.) for the desired attribute.
- Utilizing the following formula to calculate the entropy of the remaining attributes:

$$\text{Entropy}(s) = \sum_{i=1}^{n} -P_i \log_2 P_i \tag{7}$$

- The Gain (G) of each attribute is calculated by deducting the entropy(s) from the Information Gain (I.G).

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{i=1}^{n} \left(\frac{S_v}{S} \times \text{Entropy}(S_v) \right) \tag{8}$$

Feature vector data points from TF-IDF, Word2Vecand mBERT are used to build the decision tree. Important textual data, like word frequency (TF-IDF), semantic relationships (Word2Vec), contextual and multilingual semantics (mBERT), are encapsulated in these feature vectors and provide the basis for the construction of decision trees. In a decision tree, each internal node denotes a "decision" or test on a feature (such as the presence or weight of a particular term), each branch denotes the test's result, and each leaf node denotes a class label (such as fake or real news). The model is similar to a flowchart.

3.6.4 Logistic regression—LR

Logistic regression is a core classification algorithm that predicts the likelihood of a binary outcome using one or more input variables. Jr et al. (2013). Unlike linear regression, it applies the logistic (sigmoid) function to map predictions to probabilities between 0 and 1, making it suitable for binary classification problems (Mohamed and Mahmood 2025). The algorithm estimates parameters using maximum likelihood estimation and outputs the likelihood of the target class. Its simplicity, efficiency, and ability to provide interpretable coefficients make logistic regression a go-to choice for tasks such as medical diagnosis, risk assessment, and customer churn prediction (Jain et al. 2022). It estimates the probability of a sample being part of a specific class by utilizing the logistic function:

$$P(y = 1|X) = \frac{1}{1 + e^{-(w^T X + b)}} \tag{9}$$

where w represents the weight vector, X denotes the input feature vector, and b signifies the bias term. The decision rule for classification is defined as:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|X) \geq 0.5, \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

3.6.5 Naive bayes—NB

This classifier has been employed since it is most appropriate for contextual data and the dataset used, Based on the Bayes theorem's basic idea. NB is the kind of classifier that uses the supervised learning algorithm (Shaikh and Patil 2020). For the ML models, it generates rapid predictions. Text classification is the optimal use for it. The Naive Bayes Classifier is employed in binary and multi-class classifications (Akhter et al. 2021). The Naive Bayes model comes in three varieties: Gaussian NB, Multinomial NB, and Binomial NB. Gaussian NB refers to a normal distribution. Multinomial NB is mostly utilized for text classification duty, where word frequency is utilized to predict results. Similar to multinomials, binomial classifiers are employed in classification tasks (Hossain et al. 2023). We have implemented the Gaussian Naive Bayes classifier in our model.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \tag{11}$$

P(A|B): The probability that event A will occur in a way that already causes event B to occur.

Naive Bayes would approach the classification by calculating the probability that a news item is false or true based on the feature vectors. Unlike decision trees that rely on majority voting, Naive Bayes applies Bayes' theorem, assuming independence between features, to determine the likelihood of each class (fake or true) given the input features. The model then classifies the news article as fake or real based on which class has the higher probability.

3.7 Ensemble methods

An Ensemble Method is a strategy that enhances the model's overall performance and generalization by integrating the outputs from multiple distinct models. The fundamental notion behind ensemble methods is that better overall predictions can be obtained by compensating for the flaws of individual models by combining the predictions of numerous models (Akhter et al. 2021; El-kenawy 2025). Two ensemble methods that we employed in our model are explained below.

3.7.1 Gradient boosting GB

Gradient boosting in machine learning is applied to regression and classification. It's an approach to boosting. It is an ensemble machine learning strategy that sequentially aggregates the predictions from several weak learners (Hossain et al. 2023). This technique's main idea is to build models one after the other, to minimize the errors made in the prior models. However, what is the best way to accomplish that? How can we lower the error? This is accomplished by creating a new model based on the residuals or errors of the old model (Rahman et al. 2023; Sraboni et al. 2021). When the goal column is continuous, the GB Regressor is used; when the problem is one of classification, the GB Classifier is employed. Only one thing separates the two: the "Loss function." In this case, by adding weak learners using gradient descent, reducing this loss function is the objective. We will have various loss functions for classification issues (log-likelihood, for example) and regression problems (mean squared error, or MSE) because it is based on a loss function.

$$\text{Probability} = \frac{e^{\log(\text{odd})}}{1 + e^{\log(\text{odd})}} \tag{12}$$

The first step to build this model is feature extraction, which converts textual input into numerical vectors using techniques like Word2Vec, mBERT, and TF-IDF. After that, the original model receives these feature vectors and uses them to generate predictions and identify any mistakes or residuals. A new model is trained to forecast residuals, integrating prior predictions with the new model to create an enhanced combined model. This process is repeated until performance no longer significantly improves, focusing on the combined model's faults to improve forecasts progressively.

3.7.2 Extreme gradient boosting—XGBoost

XGBoost is an efficient and highly flexible execution of gradient boosting, which builds an ensemble of decision trees by minimizing a loss function iteratively (Kumar et al. 2023). It introduces several enhancements, such as tree pruning, a regularization term (L1 and L2) to control overfitting, and a split-finding algorithm that efficiently handles sparse data. XGBoost also supports parallel processing, distributed computing, and GPU acceleration, making it faster than traditional boosting algorithms (Tasnim et al. 2022). The objective function is defined as:

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{13}$$

Here, $l(y_i, \hat{y}_i)$ represents the loss function (e.g., mean squared error), and $\Omega(f_k)$ is the regularization term defined as:

$$\Omega(f_k) = \gamma T + \frac{1}{2}\lambda \|w\|^2 \tag{14}$$

where T is the number of leaves in the tree, w is the weight of the leaves, γ is the complexity penalty, and λ is the regularization parameter. This regularization helps prevent overfitting and ensures model generalization.

3.7.3 Adaptive boosting—AdaBoost

AdaBoost is a boosting method that improves the accuracy of weak classifiers, typically shallow decision trees, by iteratively combining them into a weighted ensemble. After every iteration, it gives misclassified samples more weight, compelling later models to concentrate on more challenging data points. The final model aggregates the forecasts of all weak classifiers with weights proportional to their performance. While AdaBoost works well with simple learners and handles multi-class and binary classification, it is sensitive to noisy data and outliers, impacting its performance (Kaliyar et al. 2019). The weight update rule is given as:

$$w_i^{(t+1)} = w_i^{(t)} \exp\left(\alpha_t \cdot I(y_i \neq \hat{y}_i)\right) \tag{15}$$

Where $w_i^{(t)}$ is the weight of the i-th sample at iteration t, $I(y_i \neq \hat{y}_i)$ is an indicator function that equals 1 if the sample is misclassified, and α_t is the weight of the weak learner, computed as:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - e_t}{e_t}\right) \tag{16}$$

Here, e_t is the classification error of the weak learner at iteration t.

3.7.4 Light gradient boosting machine—LGBM

LightGBM is a gradient boosting framework designed for speed, efficiency, and scalability. It employs unique techniques such as histogram-based learning, which reduces memory usage, and leaf-wise tree growth, which splits trees based on maximum information gain, resulting in deeper trees with fewer splits (Kumar et al. 2025). LightGBM can handle large datasets with high dimensionality and categorical features directly, without extensive preprocessing. Its ability to distribute training across multiple machines or GPUs makes it faster and more resource-efficient than traditional methods, like XGBoost, while achieving comparable or better predictive performance (Chen 2015). The objective function of LightGBM is expressed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \lambda \sum_{j=1}^{m} \|w_j\|^2 \tag{17}$$

Where $l(y_i, \hat{y}_i)$ represents the loss function, and the regularization term penalizes the model complexity.

3.7.5 Voting classifier—VC

The capacity of ensemble learning to mix many base models to obtain greater predictive performance has made it very popular in the field of ML. A well-known ensemble technique is the Voting Classifier, which uses the collective intelligence of the crowd to produce reliable forecasts (Freund and Schapire 1996). A voting classifier is an ML technique that maximizes the probability that a given class will be the output after learning from an ensemble of many models. It can be applied to resolve problems with both regression and classification (Salamai et al. 2021). Two voting methods are used through the Ensemble Voting Classifier: "Soft" voting and "Hard" voting.

- *Soft voting:* A probability value representing the chance that in a soft voting system, each classifier provides a particular data item that is an element of the target class. After the outputs are weighted and totaled, the significance of the classifier is taken into consideration. Next, the target label having the highest total weighted probability is chosen by popular voting (Kumar et al. 2023). Showing in the equation.

$$y = \arg\max_{i} \sum_{j=1}^{m} W_j P_{ij}, \quad i \in \{0, 1\}, \quad j \in \{1, 2, \ldots, m\} \tag{18}$$

- *Hard voting:* When classes are voted on by each classifier in hard voting (also called majority voting), the class that receives the most votes wins. Simply put, the predicted label for each item's distribution mode represents the ensemble's expected target label (Ke et al. 2017). Here, the hard voting of each classifier C would be used to determine the class mark Y in the equation:

$$Y = \text{mode}\{C_1(x), C_2(x), \ldots, C_m(x)\} \tag{19}$$

In this paper, to build the voting classifier, we choose hard voting. The first step in the process is feature extraction, which turns textual input into numerical vectors using Word2Vec, mBERT, and TF-IDF. Each base model uses these vectors as its input characteristics. These feature vectors are used to separately train a variety of classifiers, including SVM, random forest, and gradient boosting. Next, based on fresh data, each model predicts using hard voting. In hard voting, the class label with the most votes is elected as the final forecast, which is decided by majority rule.

3.7.6 Stack model

Stacking is a sophisticated ensemble learning technique that merges predictions from various base models, like SVM and RF, to create a meta-model, often employing linear regression or logistic regression. The base models are trained separately, and their predictions on the training set serve as input features for the meta-model. This meta-model leverages the strengths of the base models while minimizing their individual weaknesses, resulting in a more precise overall prediction (Elsaeed et al. 2021).

$$\hat{y} = g(f_1(X), f_2(X), \ldots, f_n(X)) \tag{20}$$

Where $f_1, f_2, \ldots, f_n$ are the base models, and g is the meta-model, such as logistic regression or a neural network.

In this research, we developed a stacked ensemble model for fake news detection, employing the Random Forest, Support Vector Machine, Extreme Gradient Boosting, and Gradient Boosting as base classifiers, with Logistic Regression serving as the meta-classifier. The model integrates predictions from base classifiers to generate a final output, enhancing accuracy and robustness in fake news identification. Figure 10 represents the architecture of the proposed model.

3.8 Proposed algorithm for fake news detection

The proposed algorithm aims to detect fake news using machine learning classifiers. It processes two datasets an English dataset and a Bangla dataset. During pre-processing, null values are handled, categorical variables are converted to a numerical format using a label encoder, and the

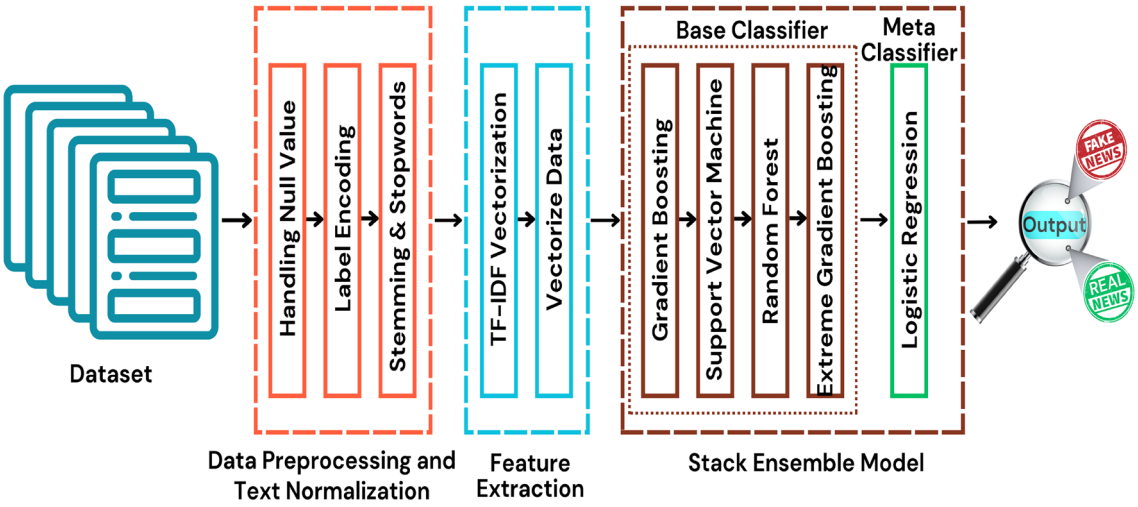


Fig. 10 Proposed architecture for stack model to detect fake news

Initialize: DS, PP, TN, FE, CL ▷ Initialize components
$DS \leftarrow$ Dataset, $X \leftarrow DS$[Content], $Y \leftarrow DS$[Label]
$PP \leftarrow$ [HNV, LE], $TN \leftarrow$ [TNZT, SWR, SL]
$FE \leftarrow$ [TF-IDF, Word2Vec, mBERT]
$CL \leftarrow$ [SVM, RF, DT, NB, GB, LR, XGB, AdaBoost, LGBM, Voting, Stack]
for $i = 1$ to $|FE|$ **do**
  $X_v \leftarrow FE[i](X)$ ▷ Vectorize X
  for $j = 1$ to $|FE|$ **do**
    Split X_v, Y into X_T, Y_T, x_t, y_t using seed j ▷ Split the Dataset
    for $k = 1$ to $|CL|$ **do**
      Train $CL[k]$ with (X_T, Y_T) ▷ Train Classifier
      Predict using $CL[k]$ on x_t ▷ Test the Classifier
      Display → Accuracy, Precision, Recall, F1 -Score ▷ Performance Metrics
    end for
  end for
end for
Deinitialize: DS, PP, TN, FE, CL ▷ Shutdown components

Algorithm 1 Fake news detection procedure

dataset is partitioned into training and testing sets. Text normalization involves tokenization, stemming or lemmatization, and stop words to reduce words to their root form. TF-IDF, mBERT, and Word2Vec representations were created for feature extraction to capture word significance and context. Eleven machine learning classifiers, SVM, RF, DT, NB, GB, LR, XGB, LGBM, Adaboost, Stacking, and a Voting Classifier, are then initialized, trained, and used to predict the test data, with performance evaluated using metrics such as recall, accuracy, precision, and F1-score. Algorithm 1 outlines the step-by-step procedures of the fake news detection system.

3.9 Experimental setup

3.9.1 Environmental setting

Experiments were conducted using an X64 processor with an Intel(R) Core i5-8265U CPU, running Windows 11 and operating at 1.60–3.90 GHz. The processor was outfitted with 12 GB of DDR4 RAM. Google Colab and Jupyter Notebook were used to run machine learning and natural language processing applications. Testing the suggested model takes around an hour, and training it takes more than sixteen hours. This extended duration is due to the use of three datasets, with SMOTE applied to one of them, and the need to run each experiment across three different feature extraction techniques: TF-IDF, Word2Vec, and mBERT. These combinations require multiple training cycles, extending the total

Table 3 Parameters of different ML models

Classifier	Parameters
SVM	(probability=True, random_state=42)
RF	(n_estimators=100, random_state=42)
DT	(max_depth=10, random_state=42)
LR	(max_iter=500, random_state=42)
GB	(n_estimators=100, random_state=42)
XGB	(use_label_encoder=False, eval_metric='mlogloss')
Adaboost	(n_estimators=50, random_state=42)
LightGBM	(n_estimators=100, learning_rate=0.1, random_state=42)

Table 4 Confusion matrix for real news and fake news classification

	Predicted real news	Predicted fake news
Actual real news	TN	FN
Actual fake news	FP	TP

training time to over sixteen hours. Additionally, the use of five-fold cross-validation and complex ensemble models, such as stacking and voting classifiers, contributes further to the computational load and time consumption.

3.9.2 Parameters of ML classifier

This section state the the parameter of different machine learning models which is used in this research. Each ml model is configured with specific hyperparameters such as the number of estimators, maximum depth, iteration limits, learning rates, and random states. These parameters ensure consistency and reproducibility in the experiments. The Table 3 provides a clear reference for the settings applied to each classifier, contributing to the reliability of the results.

3.9.3 Performance parameters

We have employed three datasets, each of which has undergone testing and training using various machine learning techniques. Precision, F1 score, Recall, and Accuracy were used to determine each model's performance. The following are the performance parameters-

- ***Confusion matrix:*** Although a confusion matrix is not a performance matrix, it is required to construct other performance matrices. The classifier's traditional way to describe performance on a test data set whose true values are known is through a table called a confusion matrix. It makes it possible to see how an algorithm performs visually [15]. A confusion Matrix is shown in Table 4.

Where,

$$\begin{aligned}\text{TN} &= \text{Machine predicted Fake and it's actually Fake}\\ \text{FN} &= \text{Machine predicted Fake and it's actually Real}\\ \text{FP} &= \text{Machine predicted Real and it's actually Fake}\\ \text{TP} &= \text{Machine predicted Real and it's actually Real}\end{aligned}$$

- ***Accuracy:*** When calculating the percentage of real or fake observations that are accurately anticipated, accuracy is frequently the most commonly utilized metric. The accuracy of an algorithm's performance can be evaluated by using the formula 12:

$$\text{Accuracy} = \left(\frac{TN+TP}{(FN+TN)+(TP+FP)}\right) \times 100 \tag{21}$$

- ***Precision:*** The precision score is the proportion of all instances that are forecasted as true to true positives. Among all the articles with positive predictions (true), precision in our instance indicates how many articles are identified as true. Equation 13 can be utilized to compute precision:

$$\text{Precision} = \left(\frac{TP}{FP+TP}\right) \times 100 \tag{22}$$

- ***Recall:*** Total positive classifications that do not belong in the actual class are represented by the recall. For us, it means how many of the entire count of true articles are anticipated to be true. Recall can be calculated utilizing the Eq. 14 following:

$$\text{Recall} = \left(\frac{TP}{FN+TP}\right) \times 100 \tag{23}$$

- ***F1 Score:*** The precision versus recall trade-off is demonstrated by the F1-score. Between each of the two, the harmonic mean is computed. For us, it takes into account both false negative and positive results. The F1-score can be identified utilizing the following formula 15.

$$\text{F1 Score} = 2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}} \times 100 \tag{24}$$

- ***ROC curve (receiver operating characteristic curve):*** The performance of a classifier across all classification thresholds is represented graphically by the Receiver Operating Characteristic curve, or ROC curve. The True Positive Rate (TPR), which is defined as follows,

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{25}$$

and FPR is defined as:

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}} \tag{26}$$

The sensitivity vs. specificity trade-off is evaluated with the aid of the ROC curve. The overall capacity of the classifier to differentiate between classes is shown by the area under the ROC curve (AUC). The model performs better when its AUC is higher.

Table 5 Performance comparison using different feature vectorization techniques and classifiers for Dataset-1

FV techniques	ML classifier	Accuracy	Precision	Recall	F1-score
TF-IDF	SVM	98.6%	98.6%	98.6%	98.6%
	RF	98.2%	98.2%	98.2%	98.2%
	DT	99.0%	99.0%	99.0%	99.0%
	LR	97.8%	98.0%	98.0%	98.0%
	GB	99.5%	99.6%	99.4%	99.4%
	Gaussian NB	84.6%	84.6%	84.6%	84.6%
	XGB	99.5%	99.4%	99.4%	99.4%
	Adaboost	99.6%	99.6%	99.6%	99.6%
	LightGBM	99.4%	99.4%	99.4%	99.4%
	Voting classifier	99.5%	99.5%	99.5%	99.5%
	Stack model	**99.6%**	**99.8%**	**99.8%**	**99.8%**
Word2Vec	SVM	98.8%	98.7%	98.7%	98.7%
	RF	97.8%	97.8%	97.8%	97.8%
	DT	94.6%	94.6%	94.6%	94.6%
	LR	**98.8%**	**99.0%**	**99.0%**	**99.0%**
	GB	98.0%	98.0%	98.0%	98.0%
	Gaussian NB	93.0%	93.0%	93.0%	93.0%
	XGB	98.2%	98.2%	98.2%	98.2%
	Adaboost	97.2%	97.0%	97.0%	97.0%
	LightGBM	98.1%	98.0%	98.0%	98.0%
	Voting classifier	98.8%	98.6%	98.6%	98.6%
	Stack Model	98.6%	98.6%	98.6%	98.6%
mBERT	SVM	89.4%	87.6%	85.6%	86.3%
	RF	97.8%	97.8%	97.8%	97.8%
	DT	95.6%	94.4%	94.0%	95.4%
	LR	**99.4%**	**99.6%**	**99.6%**	**99.6%**
	GB	89.2%	87.6%	84.4%	85.8%
	Gaussian NB	77.2%	74.2%	79.4%	74.4%
	XGB	95.8%	94.2%	95.0%	94.6%
	Adaboost	85.2%	82.0%	79.4%	80.4%
	LightGBM	95.5%	94.0%	95.0%	94.4%
	Voting Classifier	90.6%	89.0%	86.8%	87.6%
	Stack Model	99.3%	99.2%	99.2%	99.2%

4 Results analysis and discussion

In this section of the paper, we analyze the results of the proposed model. Sections 4.1–4.4 present the experimental results, including the analysis of the confusion matrix and ROC curve for each of the models tested. Section 4.5 provides an overall discussion of the proposed work, comparing it with existing approaches. Additionally, this subsection highlights the limitations of the current study and suggests areas for future improvement.

4.1 Result analysis

The findings of the proposed model utilizing Word2Vec, mBERT and TF-IDF Bi-gram as feature extraction methods are presented in this section. Accuracy, recall, F1-score, and precision are utilized to analyze the efficiency of our proposed model. We used eleven distinct machine learning classifiers to determine fake news: SVM, RF, DT, LR, NB, GB, XGB, LGBM, Adaboost, VC, and the Stacking algorithm. To increase the model's efficacy, we employed the K-Fold cross-validation technique. All the outcomes mentioned in this paper are the mean results of the K-Fold (5-fold) method. Based on dataset-1, the best result was achieved by the Stacking model with 99.6% accuracy using TF-IDF, followed by Word2Vec + Logistic Regression (98.8%) and mBERT + Logistic Regression (99.4%). For Dataset-2, before SMOTE, the Stacking model with TF-IDF achieved 74.8%, SVM with Word2Vec reached 74.2%, and mBERT + Stacking scored 72.8%. After applying SMOTE, performance improved, with Stacking + TF-IDF reaching 85.2%, Word2Vec at 79.9%, and mBERT achieving 79.2%. On Dataset-3, the best result was again with TF-IDF + Stacking (98.4%), followed by Word2Vec (87.7%) and mBERT (96.1%). We have covered the outcomes of all models using TF-IDF Bi-gram, mBERT and Word2Vec for all datasets in this section.

4.1.1 Analysis of the result for dataset-1

The Table 5 summarizes the performance of ML classifiers applied to Dataset-1, using TF-IDF Bigram, Word2Vec, and mBERT as feature vectorization techniques across accuracy, precision, recall, and F1-score. For TF-IDF Bigram, the Stack model achieved the highest performance with an accuracy of 99.6%, and 99.8% F1-score, followed closely by ensemble methods like Voting and classifiers such as

Gradient Boosting, XGB, and Adaboost. For Word2Vec, the Stack Model again stands out with the best results with 98.8% accuracy and an F1-score of 99.0%, demonstrating its consistent superiority. Although Word2Vec combined with SVM also achieved 98.8% accuracy, it lagged in other metrics such as precision, recall, and F1-score. Additionally, mBERT embeddings combined with the Logistic regression achieved 99.4% accuracy and 99.6% F1-score, followed by the stack model with an accuracy of 99.3% and F1-score of 99.2%, marking a strong performance in the contextual embedding setting. These findings highlight that the TF-IDF Bigram-based Stack Model delivers the most robust performance for dataset 1, offering marginally better results than Word2Vec and mBERT across most metrics.

Table 6 Performance comparison of different feature vectorization techniques and classifiers on Dataset-2 before applying SMOTE

FV techniques	ML classifier	Accuracy	Precision	Recall	F1-score
TF-IDF	SVM	71.0%	66.0%	56.0%	54.2%
	RF	72.2%	69.0%	59.4%	59.0%
	DT	68.4%	62.8%	62.6%	62.6%
	LR	71.4%	66.2%	59.2%	59.2%
	GB	74.3%	77.4%	63.8%	67.2%
	Gaussian NB	71.2%	65.8%	59.4%	58.2%
	XGB	74.8%	71.6%	64.4%	66.0%
	Adaboost	73.6%	70.6%	61.6%	62.0%
	LightGBM	74.7%	71.4%	65.4%	66.0%
	Voting Classifier	73.6%	70.0%	60.0%	60.0%
	Stack model	**74.8%**	**73.4%**	**66.4%**	**68.0%**
Word2Vec	**SVM**	**74.2%**	**77.0%**	**61.4%**	**63.0%**
	RF	72.4%	67.4%	60.6%	61.0%
	DT	61.8%	56.4%	56.6%	56.4%
	LR	73.4%	71.6%	60.2%	60.0%
	GB	73.0%	70.8%	59.6%	59.2%
	Gaussian NB	61.2%	58.2%	59.4%	58.2%
	XGB	70.9%	64.6%	61.2%	61.2%
	Adaboost	72.0%	67.4%	59.4%	59.8%
	LightGBM	72.9%	68.6%	61.0%	61.0%
	Voting classifier	73.8%	70.8%	62.0%	62.6%
	Stack model	73.9%	73.0%	61.2%	61.2%
mBERT	SVM	72.2%	76.2%	55.8%	53.0%
	RF	68.6%	60.8%	54.0%	51.8%
	DT	72.2%	69.6%	58.4%	57.6%
	LR	72.2%	69.6%	58.4%	72.2%
	GB	71.6%	68.6%	56.0%	54.0%
	Gaussian NB	53.0%	55.6%	56.2%	52.2%
	XGB	67.6%	59.0%	56.0%	56.0%
	Adaboost	69.6%	61.8%	56.2%	55.4%
	LightGBM	69.0%	63.4%	56.4%	55.4%
	Voting	72.4%	71.2%	57.4%	55.8%
	Stack model	**72.8%**	**72.0%**	**59.0%**	**58.4%**

4.1.2 Analysis of results for Dataset-2 without using SMOTE

Table 6 presents the performance of ML classifiers for Dataset-2 without using SMOTE, comparing TF-IDF, Word2Vec, and mBERT as feature vectorization techniques across accuracy, precision, recall, and F1-score. For TF-IDF, the Stacking Model achieved the highest performance with an accuracy of 74.8% and an F1-score of 68%, outperforming individual classifiers such as XGB and LightGBM. For Word2Vec, the Support Vector Machine recorded the best accuracy at 74.2% and an F1-score of 63%, closely followed by the Stacking Model, which obtained 73.9% accuracy and 61.2% F1-score. In the case of mBERT, the Stacking Model again stood out with the highest accuracy of 72.8% and an F1-score of 58.4%, followed by SVM, DT, LR, and Voting classifier, while other classifiers showed comparatively lower scores. These findings highlight that, although mBERT lags slightly behind in raw performance, it still provides a viable contextual embedding alternative. Among the three techniques, the TF-IDF-based Stacking Model continues to deliver the most robust results for Dataset-2 before using SMOTE, with Word2Vec and mBERT offering moderately strong alternatives depending on classifier choice.

4.1.3 Analysis of results for Dataset-2 using SMOTE

The Table 7 presents a performance comparison of various machine learning classifiers using TF-IDF, Word2Vec, and and mBERT as feature vectorization techniques for Dataset-2 after balancing the dataset using SMOTE. The performances are assessed across accuracy, precision, recall, and F1-score. For TF-IDF, the Stack Model achieved the best performance with an accuracy of 85.2% and an F1-score of 85%, followed by SVM and the Voting Classifier. In contrast, for Word2Vec, the Stack Model again outperformed other classifiers with an accuracy of 79.9% and F1-score of 80%, with Random Forest showing competitive results among individual classifiers. For mBERT, the Stack Model achieved the highest scores among the contextual embedding-based models, with an accuracy of 79.2% and an F1-score of 79.8%. These findings highlight that, while Word2Vec mBERT demonstrates strong performance under balanced conditions, the TF-IDF-based Stack Model remains the most effective overall. TF-IDF + Stack Model is our best model for identifying fake news in Bangla.

Table 7 Performance comparison of different feature vectorization techniques and classifiers on Dataset-2 after applying SMOTE

FV techniques	ML classifier	Accuracy	Precision	Recall	F1-score
TF-IDF	SVM	84.1%	84.0%	84.0%	84.0%
	RF	82.2%	82.6%	82.6%	82.6%
	DT	72.6%	72.6%	72.6%	72.6%
	LR	77.1%	77.4%	77.4%	77.4%
	GB	79.1%	79.2%	79.2%	79.2%
	Gaussian NB	74.2%	74.2%	74.2%	74.2%
	XGB	81.5%	81.5%	81.5%	81.5%
	Adaboost	75.8%	75.8%	75.8%	75.8%
	LightGBM	83.2%	83.0%	83.0%	83.0%
	Voting classifier	84.3%	84.0%	84.0%	84.0%
	Stack model	**85.2%**	**85.0%**	**85.0%**	**85.0%**
Word2Vec	SVM	66.2%	66.8%	66.4%	66.2%
	RF	79.9%	80.0%	80.0%	80.0%
	DT	67.1%	67.2%	67.2%	67.2%
	LR	61.8%	61.8%	61.8%	61.8%
	GB	65.8%	66.0%	65.8%	65.4%
	Gaussian NB	58.7%	58.6%	58.6%	58.6%
	XGB	75.5%	75.6%	75.6%	75.6%
	Adaboost	61.6%	61.4%	61.4%	61.4%
	LightGBM	73.2%	73.2%	73.2%	73.2%
	Voting classifier	74.6%	74.6%	74.6%	74.6%
	Stack model	**79.9%**	**80.0%**	**80.0%**	**80.0%**
mBERT	SVM	65.8%	65.8%	65.8%	65.8%
	RF	78.6%	78.6%	78.6%	78.6%
	DT	63.6%	63.6%	63.6%	63.0%
	LR	65.4%	65.4%	65.4%	65.4%
	GB	67.4%	67.4%	67.4%	67.4%
	Gaussian NB	56.8%	58.2%	56.8%	55.0%
	XGB	76.8%	76.8%	76.8%	76.8%
	Adaboost	62.6%	62.6%	62.6%	62.6%
	LightGBM	74.0%	74.0%	74.0%	74.0%
	Voting	70.2%	70.2%	70.2%	70.2%
	Stack model	**79.2%**	**79.8%**	**79.2%**	**79.8%**

Table 8 Performance comparison of different feature vectorization techniques and classifiers on Dataset-3

FV techniques	ML classifier	Accuracy	Precision	Recall	F1-score
TF-IDF	SVM	95.6%	94.5%	93.8%	94.4%
	RF	98.1%	98.6%	96.8%	97.6%
	DT	93.2%	92.6%	89.6%	91.0%
	LR	93.2%	92.4%	89.6%	91%
	GB	92.7%	93.4%	87.6%	90.0%
	Gaussian NB	86.6%	82.8%	89.0%	84.8%
	XGB	98.1%	97.6%	97.8%	97.8%
	Adaboost	90.6%	88.9%	86.8%	87.8%
	LightGBM	98.2%	97.6%	97.6%	97.6%
	Voting classifier	97.2%	96.8%	95.8%	96.4%
	Stack model	**98.4%**	**98.0%**	**98.0%**	**98.0%**
Word2Vec	SVM	92.6%	91.0%	89.8%	90.6%
	RF	97.0%	95.8%	96.8%	96.8%
	DT	92.4%	90.8%	89.6%	90.2%
	LR	92.4%	90.8%	89.6%	90.2%
	GB	92.8%	91.6%	90.0%	90.6%
	Gaussian NB	84.5%	80.6%	86.0%	82.0%
	XGB	97.5%	96.6%	97.6%	97.0%
	Adaboost	91.3%	89.0%	88.6%	88.8%
	LightGBM	97.4%	96.2%	97.6%	96.2%
	Voting classifier	93.5%	92.2%	91.6%	91.6%
	Stack model	**97.7%**	**97.0%**	**97.4%**	**97.8%**
mBERT	SVM	89.7%	87.6%	85.6%	86.6%
	RF	95.5%	94.6%	94.0%	94.8%
	DT	89.0%	87.0%	84.6%	85.6%
	LR	89.0%	87.0%	84.4%	84.4%
	GB	89.2%	87.6%	84.6%	85.8%
	Gaussian NB	77.2%	74.2%	79.6%	74.4%
	XGB	95.8%	94.8%	95.0%	94.6%
	Adaboost	85.2%	82.0%	79.6%	80.6%
	LightGBM	95.5%	94.0%	95.0%	94.8%
	Voting	90.6%	89.0%	86.8%	87.8%
	Stack model	**96.1%**	**95.4%**	**94.8%**	**95.6%**

4.1.4 Analysis of results for Dataset-3

Table 8 illustrates the performance outcomes of multiple machine learning classifiers applied to Dataset-3, using TF-IDF, Word2Vec, and mBERT as feature vectorization techniques. Across all metrics, accuracy, precision, recall, and F1-score, the Stacking Model consistently delivered the strongest results for each representation method. With TF-IDF, it achieved the highest scores (98.4% accuracy and 98.0% F1-score), narrowly outperforming other individual models such as XGB, LGBM, and RF. Under the Word2Vec representation, the Stacking approach again led the pack with 97.7% accuracy and a 97.8% F1-score, closely followed by LGBM and XGB. This further emphasizes its reliability when handling dense vector embeddings. Although the overall performance of mBERT embeddings was slightly lower than the other two methods, the Stacking Model still came out on top with 96.1% accuracy and a 95.6% F1-score. Notably, classifiers such as LightGBM and XGB also performed competitively, highlighting the effectiveness of contextual embeddings in conjunction with traditional machine

learning models. Overall, while all three vectorization strategies benefited from ensemble-based approaches, TF-IDF combined with Stacking yielded the best results for Dataset-3, with Word2Vec close behind, and mBERT offering a strong alternative in contextual feature representation.

4.2 Comparing overall performance

After applying two different feature extraction techniques to eleven distinct Machine Learning classifiers, in this part, we will compare the overall performance of both Bangla and English datasets and compare our model's Accuracy using different bar charts.

4.2.1 Accuracy comparison for Dataset-1

Figure 11 compares the accuracy of machine learning models using TF-IDF, Word2Vec, and mBERT for Dataset-1. Across most classifiers, TF-IDF shows slightly higher accuracy compared to Word2Vec and mBERT; the Stack Model achieved the best accuracy for TF-IDF (99.6%), and the logistic regression achieved the best accuracy for Word2Vec (98.8%) , while mBERT also demonstrated strong performance, with the Stacking model achieving 99.4%. Classifiers like Gradient Boosting, XGB, and Voting Classifier also exhibit strong accuracy for TF-IDF. Word2Vec generally performs marginally lower across models, with notable drops in classifiers like Naive Bayes, which achieved only 84.6% accuracy. mBERT showed competitive performance in several classifiers but also suffered noticeable accuracy drops in models such as GNB (77.2%) and AdaBoost (85.2%). This analysis reinforces the superiority of the TF-IDF-based Stack Model, particularly for dataset 1.

4.2.2 Accuracy comparison for Dataset-2 before using SMOTE

Figure 12 compares the accuracy of machine learning models for Dataset-2 before applying SMOTE, using TF-IDF, Word2Vec, and mBERT as feature vectorization techniques. While all three techniques yield comparable results, TF-IDF generally performs slightly better across most classifiers. The Stack Model demonstrates the highest accuracy for TF-IDF, which achieved 74.8% accuracy, and the support vector machine achieved the highest for Word2Vec, which reached 74.2% accuracy , while mBERT's best result was observed in the stacking model, achieving 72.8% accuracy. Notable variations include classifiers like Naive Bayes, where TF-IDF outperforms Word2Vec and mBERT significantly. For instance, mBERT yielded only 53.0% accuracy in the GNB model, marking substantial performance drops. Overall, the results highlight that TF-IDF-based models, especially the Stack Model, consistently achieved superior accuracy in dataset 2 without oversampling.

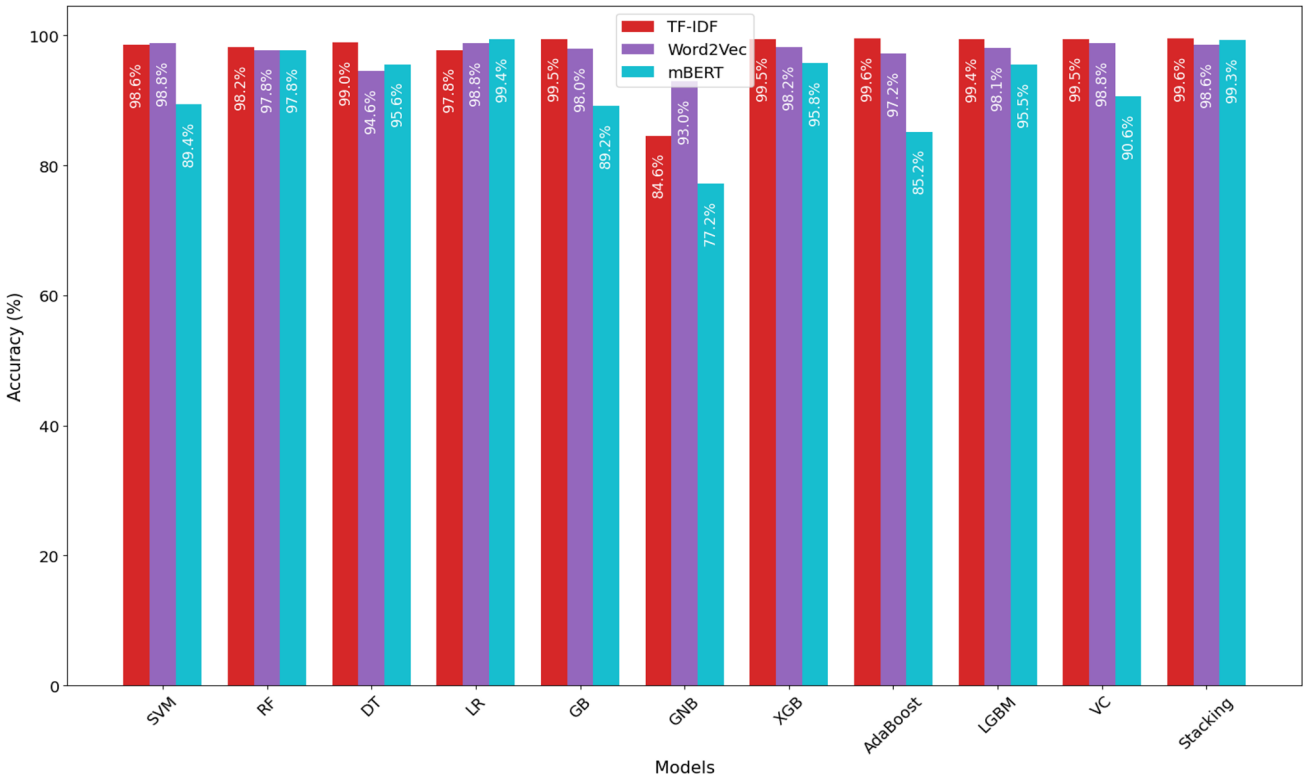


Fig. 11 Comparing the accuracy of TF-IDF, Word2Vec, and mBERT for all models for Dataset-1

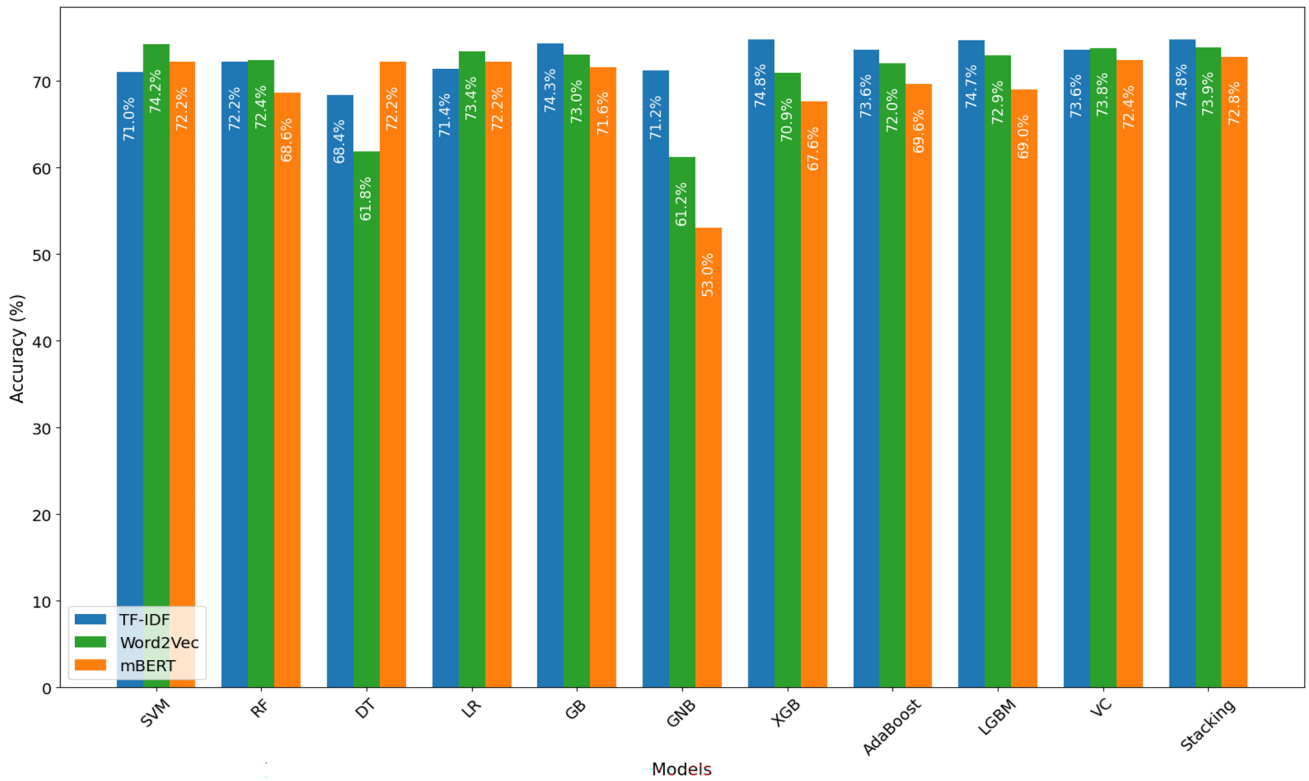


Fig. 12 Comparing the accuracy of TF-IDF, Word2Vec, and mBERT for all models for Dataset-2 before balancing the dataset using SMOTE

4.2.3 Accuracy comparison for Dataset-2 After using SMOTE

Figure 13 illustrates the accuracy of machine learning models for Dataset-2 after applying SMOTE, using TF-IDF, Word2Vec, and mBERT as feature vectorization techniques. While all three methods exhibit varying levels of effectiveness across classifiers, TF-IDF consistently achieves the highest overall accuracy. The Stack Model demonstrates the best performance across all three techniques, achieving 85.2% with TF-IDF, 79.9% with Word2Vec, and 79.2% with mBERT. Among individual classifiers, Random Forest and XGBoost perform notably well with TF-IDF, reaching 82.2% and 81.5%, respectively. Word2Vec and mBERT also show competitive results in some ensemble models, with mBERT achieving 76.8% in XGBoost and 74.0% in LGBM. However, mBERT underperforms in specific models such as Gaussian Naive Bayes, Adaboost, and Decision Trees, with accuracies of just 56.8%, 62.6%, and 63.6%, respectively, indicating weaker compatibility with simpler classifiers. These results confirm the effectiveness of TF-IDF-based Stack models, particularly when performed with SMOTE, for enhancing classification accuracy in an imbalanced dataset, while also demonstrating that mBERT has potential in complex ensemble methods but is less robust in basic classification models.

4.2.4 Accuracy comparison for Dataset-3

Figure 14 illustrates the accuracy of machine learning models for Dataset-3 using TF-IDF, Word2Vec, and mBERT as feature vectorization techniques. TF-IDF consistently achieves the highest accuracy across nearly all classifiers. The Stack Model achieves the top performance with TF-IDF at 98.4%, followed closely by Word2Vec (97.7%) and mBERT (96.1%). Individual classifiers such as XGBoost and LGBM also perform notably well with TF-IDF, reaching 98.1% and 98.2%, respectively. Word2Vec maintains high performance in ensemble models, while mBERT shows strong results in classifiers like XGBoost (95.8%) and LGBM (95.5%). Nonetheless, mBERT exhibits reduced performance in simpler models such as Gaussian Naive Bayes (77.2%) and Adaboost (85.2%) compared to the other methods. These results confirm that TF-IDF remains a robust feature representation approach, particularly when used with ensemble classifiers. At the same time, mBERT demonstrates strong potential in complex models but is less reliable in basic classifiers.

4.3 ROC curve and confusion matrix representation

In this section, we present the ROC curves and confusion matrices to provide a deeper evaluation of model

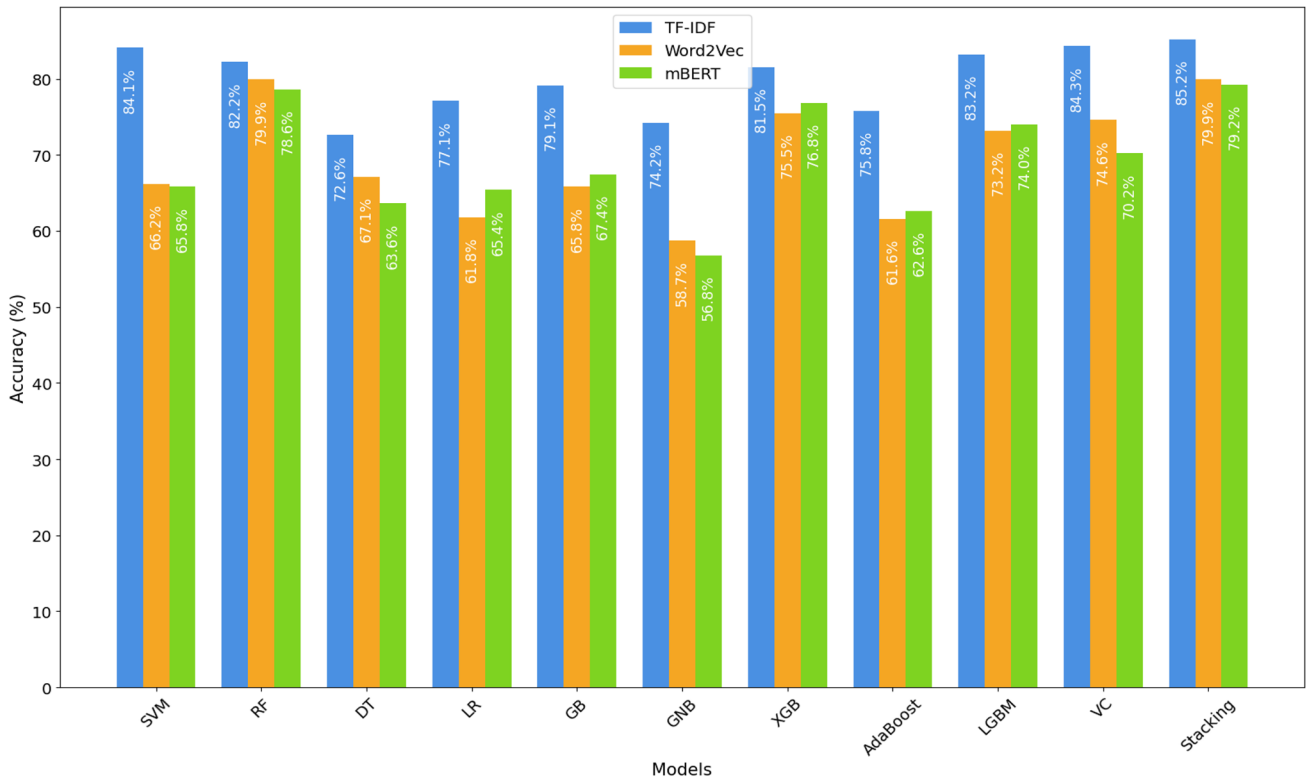


Fig. 13 Comparing the accuracy of TF-IDF, Word2Vec, and mBERT for all models for Dataset-2 after balancing the dataset using SMOTE

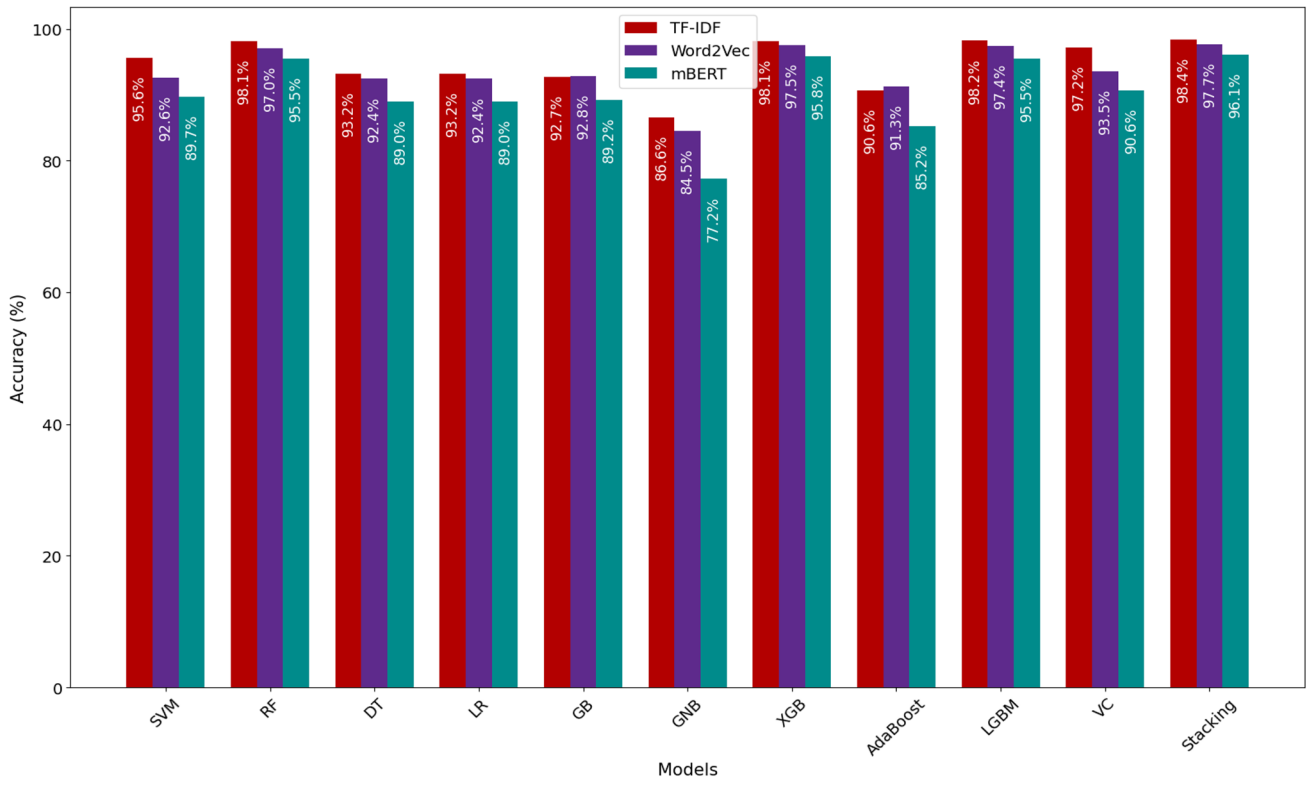
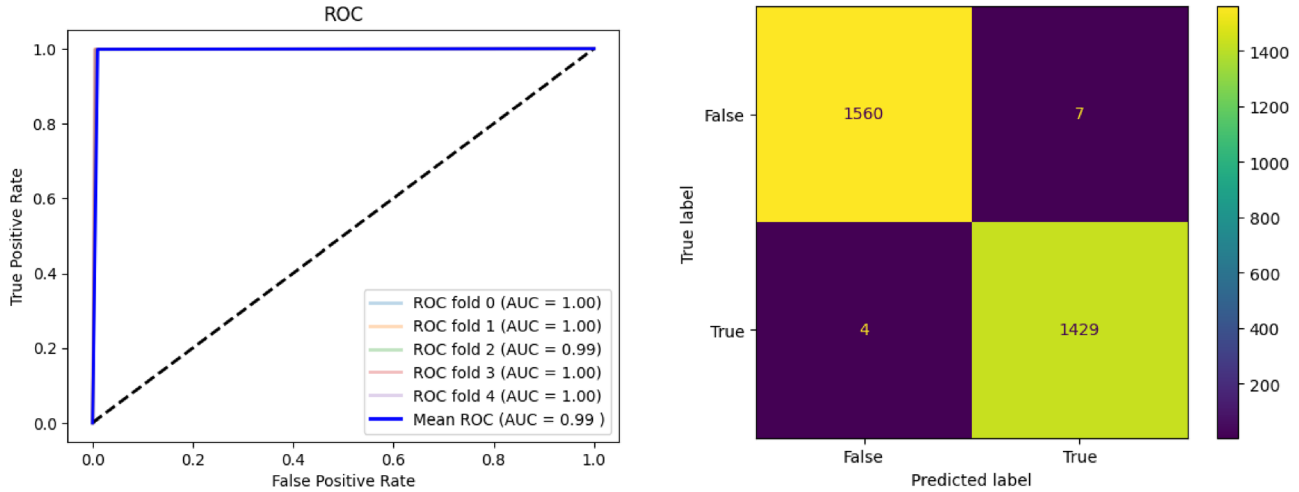
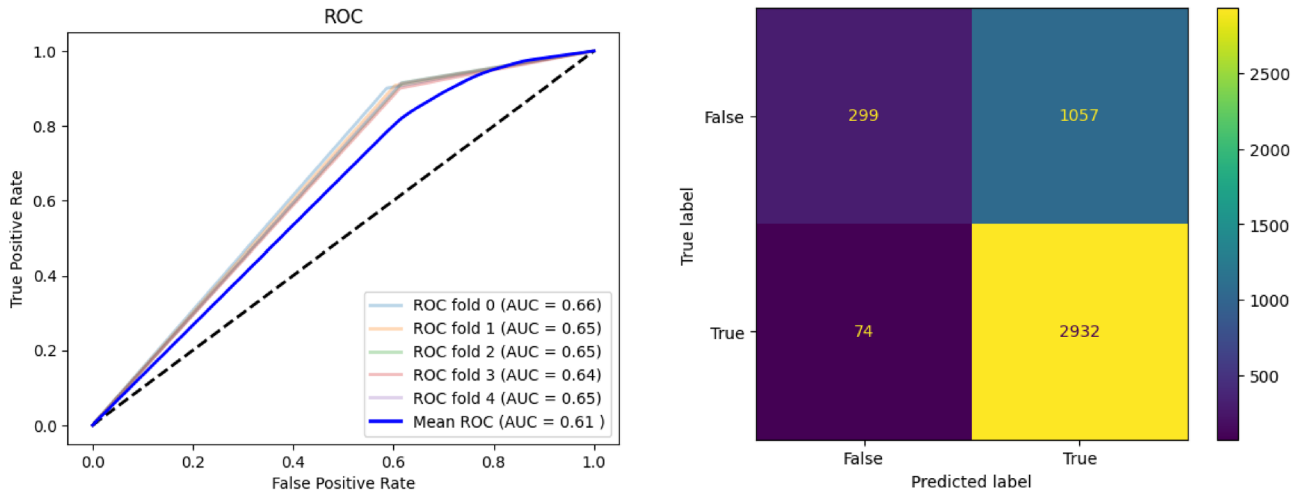


Fig. 14 Comparing the accuracy of TF-IDF Word2Vec, and mBERT for all models for Dataset-3



(a) ROC Curve (b) Confusion Matrix

Fig. 15 ROC curve and confusion matrix of TF-IDF + stack model for Dataset-1



(b) ROC Curve (b) Confusion Matrix

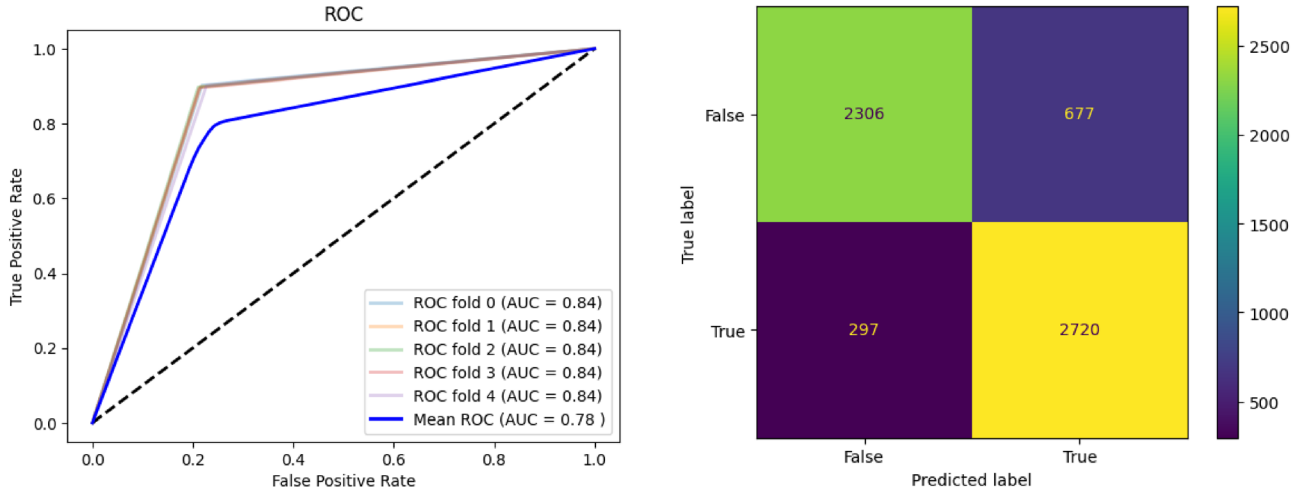
Fig. 16 ROC curve and confusion matrix of TF-IDF + stack model before balancing the Dataset-2 using SMOTE

performance. As identified in the above result analysis and comparisons, the combination of TF-IDF and the Stacking model consistently delivered the highest accuracy across all three datasets. Therefore, we focus exclusively on this best-performing configuration and display its ROC curves and confusion matrices for each dataset.

Figure 15 represents the ROC Curve and Confusion Matrix of TF-IDF + Stack Model for Dataset 1. The Stack model is the best classifier for Dataset 1 when using TF-IDF bigram, achieving an accuracy of 99.6%. The ROC curve indicates a mean ROC of 99%. The Confusion Matrix reveals high true negative (1560) and true positive (1429) values. In contrast, the false negative (7) and false positive (4) values are comparatively low. This indicates that the model performed well in this scenario.

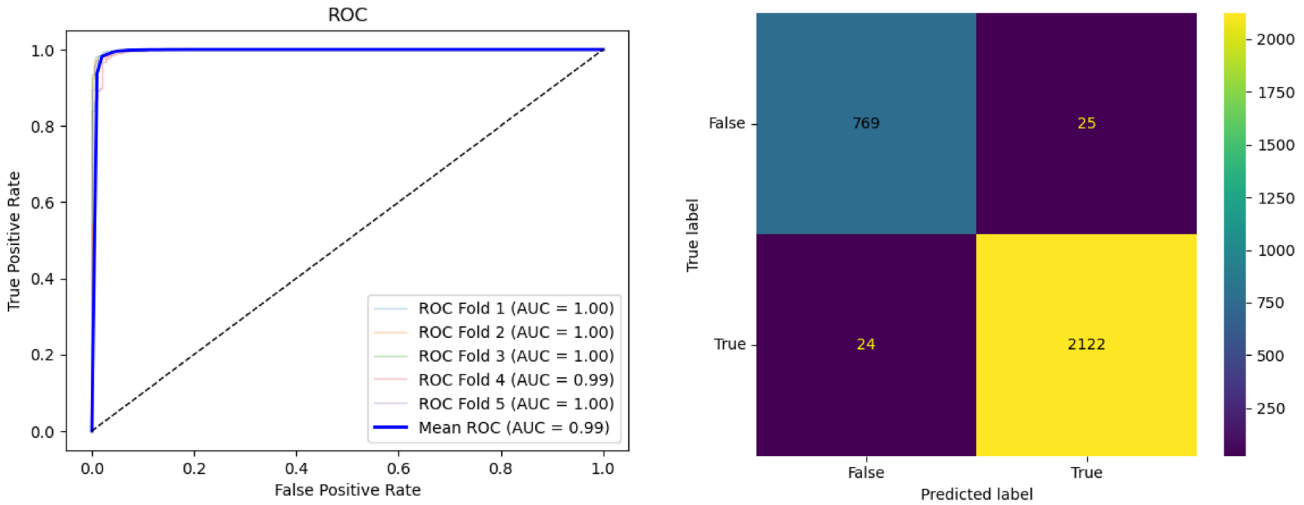
Figure 16 presents the ROC curve and confusion matrix for the TF-IDF combined with the stack model for Dataset-2 before using SMOTE to balance the dataset. The ROC curve showcases the model's performance across five folds, achieving an average AUC score of 0.61. The confusion matrix highlights the model's prediction outcomes, with 299 true negatives, 2932 true positives, 1057 false negatives, and 74 false positives. These results emphasize the model's strong predictive accuracy and robustness, particularly in identifying true positives.

Figure 17 presents the ROC curve and confusion matrix for the TF-IDF + stack model after balancing the data using



(a) ROC curve (b) Confusion Matrix

Fig. 17 ROC curve and confusion matrix of TF-IDF + stack model after balancing the Dataset-2 using SMOTE



(a) ROC curve (b) Confusion Matrix

Fig. 18 ROC curve and confusion matrix of TF-IDF + stack model for Dataset-3

SMOTE for Dataset-2. The ROC curve demonstrates the model's strong classification capability, with a mean AUC of 0.78. The Confusion Matrix reveals that the model accurately predicted 2306 true negatives and 2720 true positives, while misclassifying 677 false negatives and 297 false positives. The integration of SMOTE effectively addressed data imbalance, resulting in a more balanced class distribution that enhanced the overall model performance and improved predictive accuracy.

Figure 18 illustrates the ROC Curve and Confusion Matrix for Dataset 3, evaluated using the TF-IDF combined with the Stacked Model. This configuration achieved outstanding performance, reaching an accuracy of 98.4%. The ROC curve demonstrates a strong classification capability with a mean AUC of 99%. The Confusion Matrix highlights the model's effectiveness, with a high number of true negatives (769) and true positives (2122), while maintaining minimal false negatives (25) and false positives (24). These results demonstrate that the TF-IDF + Stacked Model configuration is highly effective and capable of delivering strong performance on Dataset 3.

Based on the analysis of the ROC curves and confusion matrices, it is evident that the TF-IDF combined with the Stacked model is the best-performing approach across all three datasets: English (Dataset 1), Bangla (Dataset 2), and the BanFakeNews (Dataset 3) dataset for fake news detection.

4.4 Computational cost and space requirement

Table 9 presents the computational cost and storage requirements of various machine learning models used for fake news detection. The table compares each model's training time (in seconds) and the required memory (in megabytes). This comparison is based only on TF-IDF-based classifiers, as TF-IDF produced the best results in our experiments. Among the models, Logistic Regression (LR) has the shortest training time (17 s) with a moderate memory requirement of 662 MB, whereas the Voting model takes the longest time (2246 s) with a memory usage of 1373 MB. The Stacked Model, which achieved the highest accuracy 99.6%, required 1821 s for training and used 1223 MB of memory. Notably, SVM had the highest memory consumption (3816 MB), making it the most space-intensive model, while Decision Tree (DT) was the most efficient in terms of storage, requiring only 57 MB. Despite the stacked model's relatively high training time, it demonstrates a balanced trade-off between performance and memory efficiency, making it the optimal choice for fake news detection.

4.5 Discussion and limitations

The whole world faces a serious problem with fake news, especially in countries like Bangladesh, India, and Pakistan. Numerous studies have been done before to eliminate or stop the spread of fake news, including the application of ML and DL. Studies on the subject of identifying fake news are now accessible and available in many journals. To prevent the spread of rumors, we developed this model to identify false news. In light of this study, an enhanced false news identification system utilizing word2vec, mBERT, TF-IDF, and eleven machine learning algorithms that were proposed. The models are trained using three publicly available datasets. In our observation, if we did not make any mistakes, no paper was published, and no work was done specifically before using the Bangla dataset (Dataset-2).

Table 9 Computational cost and space requirement for TF-IDF + ML classifiers

Model	Training time	Required space
SVM	91 s	3816 MB
RF	116 s	3835 MB
DT	27 s	57 MB
LR	17 s	66 MB
GB	634 s	1322 MB
Gaussian NB	98 s	2453 MB
XGB	109 s	3997 MB
Adaboost	235 s	1378 MB
LightXGB	62 s	1181 MB
Voting	2246 s	1373 MB
Stack model	1821 s	1223 MB

To determine the most effective model for fake news detection, we systematically compared eleven machine learning algorithms based on multiple performance parameters, including accuracy, precision, recall, F1-score, and AUC. Additionally, we assessed each model using a confusion matrix and ROC curve analysis while also considering computational cost in terms of time and space complexity. Based on our extensive evaluation, we found that TF-IDF combined with the stacked model emerged as the best-performing classifier, achieving an impressive accuracy of 99.6%. Although its time complexity is higher than other models, making it the most time-consuming approach, the stacked model requires comparatively less memory. We also integrated mBERT as a feature extraction method and classified using the same Stacking model. While mBERT-based models showed competitive performance, the TF-IDF + Stacking configuration still outperformed all others across all evaluation metrics, making it the most effective choice for fake news detection. Given the strong performance of our stacked model, we did not pursue more complex AI models, such as deep learning architectures, as they would further increase computational costs in terms of both training time and memory usage without guaranteeing significant performance improvements.

In addition, we compared our best-performing stacked model with prior research in the field of fake news detection to assess its effectiveness. As discussed in the Related Work section, several studies have explored similar approaches. Our results are evaluated against existing methodologies from previous studies, as presented in Table 10. This comparison highlights the advantages of our proposed model over traditional classifiers, demonstrating its superior performance in identifying fake news.

When we compared our English fake news detection method to the classification methods in related works, our proposed classifier produced significantly better results. Among the papers listed in Table 10, Asha et al. (2024) achieved the highest accuracy of 99%. In contrast, Mugdha et al. (2020) used Gaussian Naive Bayes (NB) to detect fake news in Bengali, achieving 87% accuracy, which was the best among their techniques. Similarly, Hussain et al. (2020) used SVM and MNBs to identify fake news in Bangla, with SVM achieving a 96.64% accuracy, the highest in their study.

When comparing our proposed Bangla fake news detection method to previous studies, the initial improvement may seem modest. This was largely due to the limited size and quality of the initial Bangla dataset, as well as the lack of essential language tools. Notably, our initial evaluation was conducted on a newly introduced dataset that had not been used in prior research. However, when we later tested our model on the well-established BanFakeNews dataset,

Table 10 Comparison of existing works with our proposed model

Refs.	Model	Algorithms/techniques	Highest accuracy
Asha et al. (2024)	ML Model	RF, NB, LR	99.06% (RF)
Holla and Kavitha (2024)	ML Model	ID-3, NB, RF, AdaBoost	98.98% (AdaBoost)
Choudhury and Acharjee (2023)	ML Model	SVM, NB, RF, LR	97% (SVM)
Altheneyan and Alhadlaq (2023)	ML + FV + Big Data	RF, DT, LR, Ensemble	93.45% (HashingTF-IDF_Ensembler)
Jain et al. (2024)	ML Model	SVM, RF, KNN, NB, LR, Bagging, Boosting	97.31% (RF)
Asha and Meenakowshalya (2021)	ML Model	DT, LR, KNN, SVM, LSVM, SGD	93.5% (LSVM)
Mugdha et al. (2020)	ML Model	Gaussian NB, SVM, LR, RF, Multinomial NB, AdaBoost, GB, Voting	87% (Gaussian NB)
Hussain et al. (2020)	ML Model	Multinomial NB, SVM	96.64% (SVM), 93.32% (Multinomial NB)
Proposed model	Feature Vectorization + ML	SVM, RF, DT, LR, GB, NB, XGB, AdaBoost, LightGBM, Voting, and Stack + (Word2Vec, mBERT, TF-IDF)	99.6% for Dataset 1, 85.2% for Dataset 2, and 98.4% for dataset-3 (Stack + TF-IDF)

it not only performed well but also outperformed existing methods, demonstrating its effectiveness on more established and structured data. While our main objective was to develop a robust model for under-resourced languages rather than to chase state-of-the-art accuracy, these results validate the strength of our approach. We aim to continue improving Bangla fake news detection by incorporating richer datasets and more advanced features in future work.

This study offers a thorough literature analysis, which is valuable to both students and researchers. By identifying the most frequently discussed topics in the fake news domain, it helps journals better understand how fake news spreads, supporting governments, funding organizations, and society in making more informed decisions about future efforts.

5 Conclusion and future work

This study combined eleven machine learning algorithms with feature extraction methods such as Word2Vec and TF-IDF to present an improved fake news detection system. The most successful model for detecting fake news is the Stack Model, in conjunction with TF-IDF. It achieved 99.6% accuracy and an F1-score of 99.8% for English fake news identification and 85.16% accuracy for Bangla fake news identification. Additionally, it achieved 98.4% accuracy on the BanFakeNews dataset, demonstrating strong performance even in a low-resource language. The system's accuracy on the Bangla dataset (85.2%) was lower than its performance on the English dataset, highlighting the challenges of fake news detection in low-resource languages, where high-quality annotated datasets are limited. Models like TF-IDF and Word2Vec primarily rely on word-level representations, which often fail to capture deeper contextual nuances such as sarcasm, implied meaning, or cultural references. This can lead to misclassifications, particularly when dealing with ambiguous or misleading content. Additionally, models trained on static datasets may struggle to generalize to emerging patterns of misinformation, as fake news content evolves rapidly. Future research could explore advanced language models like BERT, LSTM, GPT, BanglaBERT, AlBERT, DistilBERT, and RoBERTa, which excel in understanding context and semantics and may enhance fake news detection in low-resource languages like Bangla. Additionally, we plan to apply deep learning and transfer learning techniques to improve prediction accuracy across various languages, including Bangla. Developing models that can identify and classify fake news in real time as it spreads on social media is essential. This requires algorithms capable of processing streaming data efficiently and adaptively. Integrating explainability techniques would enhance the system's transparency, making it more valuable for journalists and fact-checkers. These techniques would help users understand why certain news is flagged as fake, fostering trust in the system. Furthermore, in the future we will focus on exploring cross-lingual transfer learning, conducting a detailed error analysis to categorize challenging news types, and performing ablation studies to quantify the unique contribution of our stacking ensemble.

6 Ethical consideration

While our study aims to mitigate the spread of fake news, it is essential to consider the ethical implications of automated fake news detection. Misclassifications can lead to severe consequences, including unjust censorship or the suppression of legitimate information. Furthermore, biases in datasets and models may disproportionately impact certain groups or viewpoints. Since we used publicly available datasets, we acknowledge the importance of transparency and fairness in both data collection and model evaluation. Ensuring the responsible use of AI in combating misinformation remains a critical direction for future work.

Acknowledgements The study presented here is the result of a collaborative effort by the authors. No external support was received for

this study.

Author contributions Conceptualization: Md. Sabbir Hossen, Pabon Shaha, Anichur Rahman; Methodology: Md. Sabbir Hossen, Pabon Shaha, Anichur Rahman; Formal analysis and investi- gation: Fahjimatus Sabah, K. M. Mursalin Billah Rezwan; Writing—original draft preparation: Md. Sabbir Hossen, Pabon Shaha, Md. Mowahibur Rahman Twake, and Abu Saleh Musa Miah; Writing—review and editing: Md. Sabbir Hossen, Pabon Shaha, Anichur Rahman, Fahim Al Farid, Hezerul Abdul Karim; Funding acquisition: Fahim Al Farid, Abu Saleh Musa Miah, Hezerul Abdul Karim; Resources: Fahim Al Farid, Abu Saleh Musa Miah, Fahjimatus Sabah; Supervision: Pabon Shaha, and Anichur Rahman.

Funding None reported.

Data availability The source code is available via the GitHub link. The proposed approach and schematic diagram are shared with the scientific community for public access: https://github.com/PabonSC/Fake-News-Detection.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

Abualigah L, Al-Ajlouni YY, Daoud MS, Altalhi M, Migdady H (2024) Fake news detection using recurrent neural network based on bidirectional lstm and glove. Soc Netw Anal Min 14:40

Akhter MP, Zheng J, Afzal F et al (2021) Supervised ensemble learning methods towards automatically filtering urdu fake news within social media. PeerJ Comput Sci 7:e425

Alghamdi J, Lin Y, Luo S (2023) Towards covid-19 fake news detection using transformer-based models. Knowl-Based Syst 274:110642

Alghamdi J, Luo S, Lin Y (2024) A comprehensive survey on machine learning approaches for fake news detection. Multimed Tools Appl 83:51009–51067

Alhussan AA, Khafaga DS, El-Kenawy EM, Ibrahim A, Eid MM, Abdelhamid AA (2022) Pothole and plain road classification using adaptive mutation dipper throated optimization and transfer learning for self-driving cars. IEEE Access 10:84188–84211. https://doi.org/10.1109/ACCESS.2022.3196660

Allen J, Watts DJ, Rand DG (2024) Quantifying the impact of misinformation and vaccine-skeptical content on facebook. Science 384:eadk3451

Altheneyan A, Alhadlaq A (2023) Big data ml-based fake news detection using distributed learning. IEEE Access 11:29447–29463

Asha J, Meenakowshalya A (2021) Fake news detection using n-gram analysis and machine learning algorithms. J Mob Comput Commun Mob Netw 8:33–43

Azzeh M, Qusef A, Alabboushi O (2025) Arabic fake news detection in social media context using word embeddings and pre-trained transformers. Arab J Sci Eng 50(2):923–936

Balshetwar SV, Rs A (2023) Fake news detection in social media based on sentiment analysis using classifier techniques. Multimed Tools Appl 82:35781–35811

Breiman L (2001) Random forests. Mach Learn 45:5–32

Chen T (2015) XGBoost: extreme gradient boosting. R Package Version 0.4-2

Choudhury D, Acharjee T (2023) A novel approach to fake news detection in social networks using genetic algorithm applying machine learning classifiers. Multimed Tools Appl 82:9029–9045

Coelho S, Hegde A, Kavya G, Shashirekha HL (2023) Mucs@dravidianlangtech2023: Malayalam fake news detection using machine learning approach. In: Proceedings of the Third Workshop on Speech and Language Technologies for Dravidian Languages

Devlin J (2018) BERT: pre-training of deep bidirectional transformers for language understanding. arXiv preprint, arXiv:1810.04805. https://arxiv.org/abs/1810.04805

El-kenawy EM (2025) A review of machine learning models for predicting air quality in urban areas. Metaheuristic Optim Rev 3(2):33–46

Elsaeed E, Ouda O, Elmogy MM, Atwan A, El-Daydamony E (2021) Detecting fake news in social media using voting classifier. IEEE Access 9:161909–161925

Farooq MS, Naseem A, Rustam F, Ashraf I (2023) Fake news detection in urdu language using machine learning. PeerJ Comput Sci 9:e1353

Ghamdi MAA, Bhatti MS, Saeed A, Gillani Z, Almotiri SH (2024) A fusion of bert, machine learning and manual approach for fake news detection. Multimed Tools Appl 83:30095–30112

Holla L, Kavitha K (2024) An improved fake news detection model using hybrid time frequency-inverse document frequency for feature extraction and adaboost ensemble model as a classifier. J Adv Inf Technol 15:202–211

Hosmer DW Jr, Lemeshow S (2013) Applied logistic regression. John Wiley & Sons

Hossain MM, Chowdhury ZR et al (2023) Crime text classification and drug modeling from Bengali news articles: a transformer network-based deep learning approach. In: 2023 26th International Conference on Computer and Information Technology (ICCIT). IEEE

Hossen MS, Shaha P, Saiduzzaman M (2025) Social media sentiments analysis on the July revolution in Bangladesh: a hybrid transformer based machine learning approach. In: 17th International Conference on Electronics, Computers and Artificial Intelligence (ECAI). IEEE

Hussain MG, Hasan MR, Rahman M, Protim J, Hasan SA (2020) Detection of bangla fake news using mnb and svm classifier. In: 2020 International Conference on Computing, Electronics & Communications Engineering (iCCECE). IEEE

Jain MK, Gopalani D, Meena YK (2024) Confake: fake news identification using content based features. Multimed Tools Appl 83:8729–8755

Jain P, Sharma S et al (2022) Classifying fake news detection using svm, naive bayes and lstm. In: 2022 12th International Conference on Cloud Computing, Data Science & Engineering (Confluence). IEEE

Jr DWH, Lemeshow S, Sturdivant RX (2013) Applied logistic regression. John Wiley & Sons

Kaliyar RK, Goswami A, Narang P (2019) Multiclass fake news detection using ensemble machine learning. In: 2019 IEEE 9th International Conference on Advanced Computing (IACC). IEEE

Kapusta J, Držík D, Šteflovič K, Nagy KS (2024) Text data augmentation techniques for word embeddings in fake news classification. IEEE Access 12:31538–31550

Kaur S, Kumar P, Kumaraguru P (2020) Automating fake news detection system using multi-level voting model. Soft Comput 24:9049–9069

Ke G, Meng Q, Finley T, Wang T, Chen W et al (2017) Lightgbm: a highly efficient gradient boosting decision tree. Adv Neural Inf Process Syst 30:234

Khalil M, Zhang C, Ye Z, Zhang P (2025) Pegasosqsvm: a quantum machine learning approach for accurate fake news detection. Appl Artif Intell 39(1):2457207

Kumar S, Kumar D, Singh SR (2023) Gated recursive and sequential deep hierarchical encoding for detecting incongruent news articles. IEEE Trans Comput Soc Syst 11(1):1023–1034

Kumar S, Kumar S, Singh SR (2024) A headline-centric graph-based dual context matching approach for incongruent news detection. IEEE Trans Comput Soc Syst 11(5):5913–5924

Kumar S, Agrahari S, Soni P, Sachdeva A, Singh SR (2025) Fake news detection using Hashtag context. Pattern Recognit Lett 1:1

Kumar S, Kumar M, Singh SR (2023) Language-independent fake news detection over social media networks using the centrality-aware graph convolution network. In: International Conference on Mathematics and Computing, 89–97, Singapore: Springer Nature Singapore

Lahby M, Aqil S, Yafooz WM, Abakarim Y (2022) Online fake news detection using machine learning techniques: a systematic mapping study, 3–37

Madani M, Motameni H, Roshani R (2024) Fake news detection using feature extraction, natural language processing, curriculum learning, and deep learning. Int J Inf Technol Decis Mak 23:1063–1098

Mahir SH, Tashrif MTA, Karim MA, Kundu D, Rahman A, Hamza MA, Al Farid F, Miah ASM and Mansor S (2025) Advanced hydro-informatic modeling through feedforward neural network, federated learning, and explainable AI for enhancing flood prediction. IEEE Open J Comput Soc vol. 6, pp. 726-738https://doi.org/10.1109/OJCS.2025.3556424

Mahmud T, Hasan I, Aziz MT, Rahman T, Hossain MS et al (2024) Enhanced fake news detection through the fusion of deep learning and repeat vector representations. In: 2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT). IEEE

Mallik A, Kumar S (2024) Word2vec and lstm based deep learning technique for context-free fake news detection. Multimed Tools Appl 83:919–940

Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781

Mohamed A, Mahmood S (2025) K-nearest neighbors approach to analyze and predict air quality in Delhi. J Artif Intell Metaheuristics 9(1):34–43. https://doi.org/10.54216/JAIM.090104

Mugdha SBS, Ferdous SM, Fahmin A (2020) Evaluating machine learning algorithms for Bengali fake news detection. In: 2020 23rd International Conference on Computer and Information Technology (ICCIT). IEEE

Pal A, Pradhan M (2023) Survey of fake news detection using machine intelligence approach. Data Knowl Eng 144:102118

Rabbi MSH, Bari MM, Debnath T, Rahman A, Das AK, Hossain MP, Muhammad G (2025) Performance evaluation of optimal ensemble learning approaches with PCA and LDA-based feature extraction for heart disease prediction. Biomed Signal Process Control 101:107138

Rahman A, Hossain MS, Muhammad G, Kundu D, Debnath T et al (2023) Federated learning-based AI approaches in smart healthcare: concepts, taxonomies, challenges and open issues. Clust Comput 26(4):2271–2311

Rahman A, Khan MSI, Montieri A, Islam MJ, Karim MR et al (2024) Blocksd-5gnet: enhancing security of 5g network through blockchain-sdn with ml-based bandwidth prediction. Trans Emerg Telecommun Technol 35(4):e4965

Salamai AA, El-kenawy EM, Ibrahim A (2021) Dynamic voting classifier for risk identification in supply Chain 4.0. Comput Mater Continua 69(3):3749–3766. https://doi.org/10.32604/cmc.2021.018179

Salh DA, Nabi RM (2023) Kurdish fake news detection based on machine learning approaches. Passer J Basic Appl Sci 5:262–271

Shaikh J, Patil R (2020) Fake news detection using machine learning. In: 2020 IEEE International Symposium on Sustainable Energy, Signal Processing and Cyber Security (iSSSC). IEEE

Sharma U, Saran S, Patil SM (2020) Fake news detection using machine learning algorithms. Int J Creat Res Thoughts (IJCRT) 8:509–518

Song Y-Y, Ying L (2015) Decision tree methods: applications for classification and prediction. Shanghai Arch Psychiatry 27:130

Suthaharan S (2016) Support vector machine. Machine learning models and algorithms for big data classification: thinking with examples for effective learning. 207–235

Tasnim A, Saiduzzaman M, Rahman MA, Akhter J, Rahaman ASMM (2022) Performance evaluation of multiple classifiers for predicting fake news. J Comput Commun 10:1–21